GUIDE 4 OF 4 FOR THE petri-flow REPOSITORY

PetriFlow on Azure OpenAI with a service principal

What your mentor is giving you, what has to change in the codebase (very little), the exact diff, a GitHub Copilot prompt that applies it, and the checks that prove it worked. Company laptop, Git Bash, no personal key.

Verified 9 October 2026: typecheck, mock mode, and the Entra ID path with fake credentials (AADSTS90002).



WHAT IS INSIDE

Part 1 Service principal from zero: what it is, why companies use it, what to ask your mentor
Part 2 What changes in PetriFlow and what does not (nothing under packages/)
Part 3 The design: provider switch, token flow, one request on the wire
Part 4 The changes file by file, with the diff
Part 5 The GitHub Copilot prompt and how to use it safely
Part 6 Run it: environment, Lab 0 in two stages, then the four labs
Part 7 Troubleshooting, secret hygiene, what was verified, cleanup
Appendix A: full diff. Appendix B: full files, paste-ready. Appendix C: the prompt file.

Companions: 'explained from zero' (concepts), 'run it yourself' (tests), 'with your own OpenAI key' (the labs).

Contents

Contents 2

Part 1. Service principal, from zero 4

1.1 Three things that look similar and are not 4

1.2 Why companies prefer this over a key 4

1.3 What to ask your mentor for (checklist) 4

Part 2. What changes in PetriFlow, and what does not 6

2.1 Why the repo itself needs no change 6

2.2 The change map 6

Part 3. The design of the change 8

3.1 Provider selection from the environment 8

3.2 The token flow 8

3.3 One request on the wire 9

3.4 Endpoint normalisation 9

Part 4. The changes, file by file 10

4.1 package.json: two dependencies 10

4.2 labs/openai/_model.ts: the plug 10

4.3 labs/openai/00-check-key.ts: a two-stage check 12

4.4 labs/openai/03-server.ts: five lines 13

4.5 labs/openai/04-decision-provider.ts: four lines 14

4.6 Optional: the repo's own examples on Azure 14

Part 5. The GitHub Copilot prompt 16

5.1 Before you paste 16

5.2 The prompt (also in labs/openai/COPILOT_PROMPT_AZURE.md) 16

5.3 What to check in Copilot's diff 17

Part 6. Run it 19

Part 7. Troubleshooting, hygiene, verification, cleanup 21

7.1 Error to cause to fix 21

7.2 Secret hygiene on a company laptop 21

7.3 What was verified, and what was not 21

7.4 Cleanup 22

Appendix A. The complete diff 23

Appendix B. The changed files, paste-ready 28

labs/openai/_model.ts 28

labs/openai/00-check-key.ts 30

labs/openai/03-server.ts 31

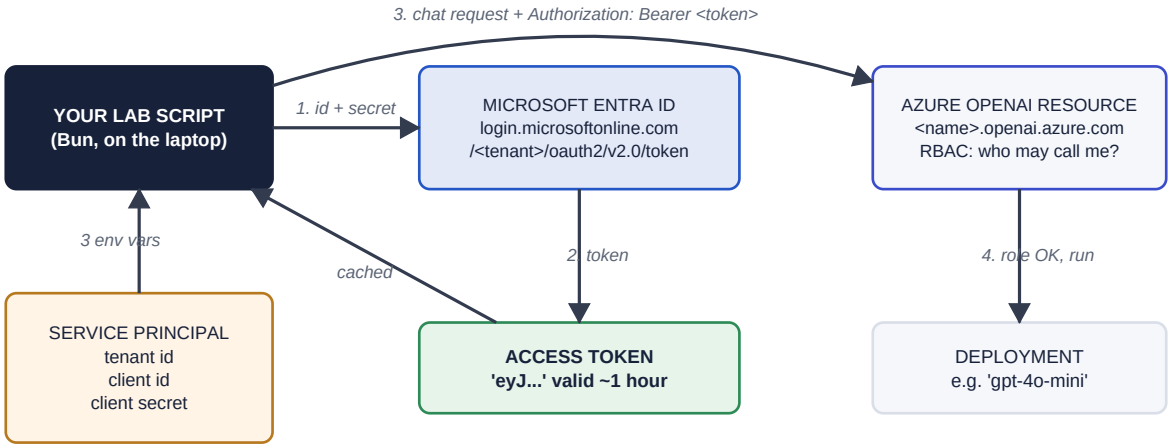
labs/openai/04-decision-provider.ts 33
labs/openai/README.md 34
Appendix C. The prompt file 35

Part 1. Service principal, from zero

Your mentor said: *no personal key; use Azure OpenAI; access it through a service principal*. This part explains those words so the rest of the guide is obvious.

1.1 Three things that look similar and are not

Thing	What it is	Analogy
OpenAI API key (sk-...)	One long secret tied to a personal or company OpenAI account. Whoever has it can spend money.	A door key. Anyone holding it gets in.
Azure OpenAI	Microsoft hosts the same model families inside your company's Azure subscription. Access is controlled by Azure, not by OpenAI accounts.	The same machine, installed in the company building.
Service principal	An identity for a program (an 'app registration' in Microsoft Entra ID, formerly Azure AD). It has a tenant id, a client id and a secret or certificate. It logs in, gets a short-lived token, and Azure checks which roles it holds.	An employee badge for a robot. Security checks the badge at every door.



An API key is one secret that unlocks the door directly. A service principal is an identity: it proves who it is, gets a short-lived token, and the resource checks a role (RBAC) before answering. The script does steps 1 to 3 by itself.

How one request reaches the model with a service principal. Steps 1 to 3 are done by two library calls.

1.2 Why companies prefer this over a key

- **Nothing permanent travels.** The token expires in about an hour. A leaked token is worth little; a leaked key is worth a monthly bill.
- **Least privilege.** The principal gets exactly one role on exactly one resource: 'Cognitive Services OpenAI User'. It cannot create resources or read other people's data.
- **Audit and rotation.** Every call is logged under the principal's name. Secrets have expiry dates and can be rotated without touching code.
- **No personal account on a managed device.** This is the point your mentor is solving.

1.3 What to ask your mentor for (checklist)

#	Item	Looks like	Env var in this guide
1	Tenant ID	GUID, e.g. 72f988bf-...-2d7cd011db47	AZURE_TENANT_ID
2	Client (application) ID	GUID	AZURE_CLIENT_ID
3	Client secret VALUE (not the secret id), plus its expiry date. Or a certificate file instead.	long random string	AZURE_CLIENT_SECRET (or AZURE_CLIENT_CERTIFICATE_PATH)
4	Endpoint of the Azure OpenAI resource	https://<name>.openai.azure.com	AZURE_OPENAI_ENDPOINT
5	Deployment name(s) and which model each one is	e.g. 'gpt-4o-mini' or 'chat-mini'	AZURE_OPENAI_DEPLOYMENT
6	Confirmation that the principal has the role 'Cognitive Services OpenAI User' on that resource	a yes, or a screenshot of IAM	(none; causes 401 if missing)
7	Is there a proxy or API gateway (APIM) in front? Does the laptop reach login.microsoftonline.com?	a URL with a path, or 'direct'	HTTPS_PROXY if needed
8	Is the Azure CLI (az) installed or allowed? Optional alternative login.	yes/no	(none)

NOTE The deployment name is the number one source of confusion

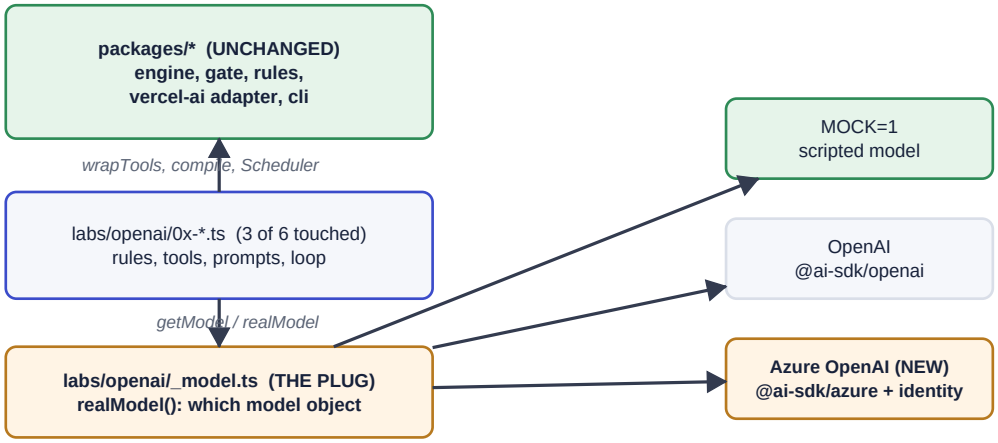
In Azure you do not call a model, you call a **deployment**: a named copy of a model inside the resource, created by an admin. The code sends that name in the 'model' field. If the admin called it 'chat-mini', then AZURE_OPENAI_DEPLOYMENT=chat-mini, even though the model behind it is gpt-4o-mini.

LOOPHOLE Secret hygiene is stricter than with a personal key

The secret belongs to the company. Export it only in the Git Bash window (`export AZURE_CLIENT_SECRET=...`), never in .bashrc, never in a file inside the repo, never in Copilot Chat, never in a ticket. If it ever lands in a file, tell your mentor so it can be rotated. `git status` before every commit.

Part 2. What changes in PetriFlow, and what does not

Short answer: **nothing under packages/**. PetriFlow is a safety layer that sits between an agent loop and its tools. The model is a plug at the far end of that chain, and the Vercel AI SDK hides every provider behind one interface called LanguageModel. Swapping OpenAI for Azure OpenAI therefore means changing the one function that builds that object.



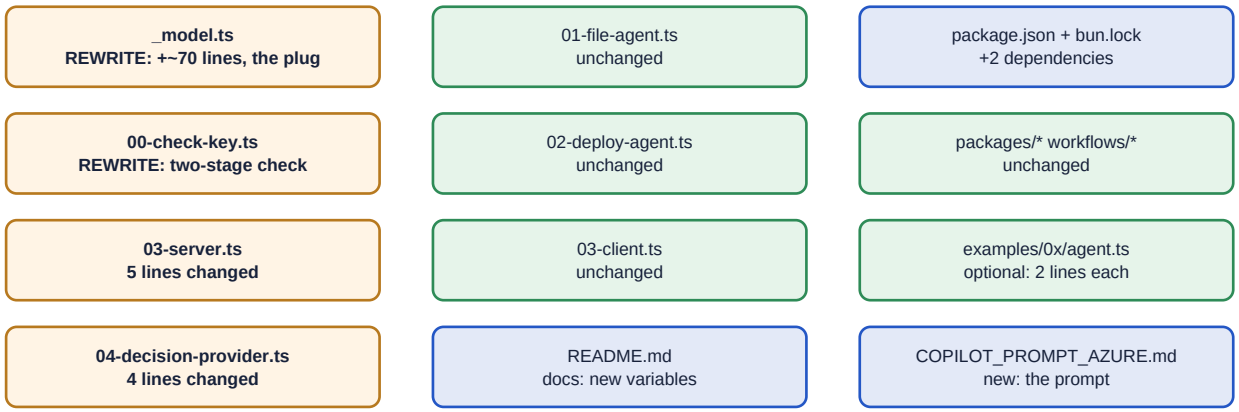
The gate only ever sees tool calls and tool results. It cannot tell which model made them. That is why the provider swap is tiny.

Where the change lives. Green is untouched, orange is new, blue is docs or dependencies.

2.1 Why the repo itself needs no change

- **The gate sees events, not models.** `createPetriflowGate(nets).wrapTools(tools)` intercepts each tool's execute function. It receives a tool name and input, consults the nets, and allows or throws. The identity of the model that asked is not part of the event.
- **The engine's decision provider is an interface.** Lab 4 implements `choose(req)`; what happens inside (OpenAI, Azure, a coin flip) is invisible to the Scheduler.
- **The repo's own examples are Anthropic-only.** They build the model with `anthropic("claude-...")`. They are not needed for the labs; Part 4.6 shows the optional two-line swap if you ever want them on Azure.

2.2 The change map



Twelve files that matter. Four edited, three untouched by design, the rest are docs, dependencies or optional.

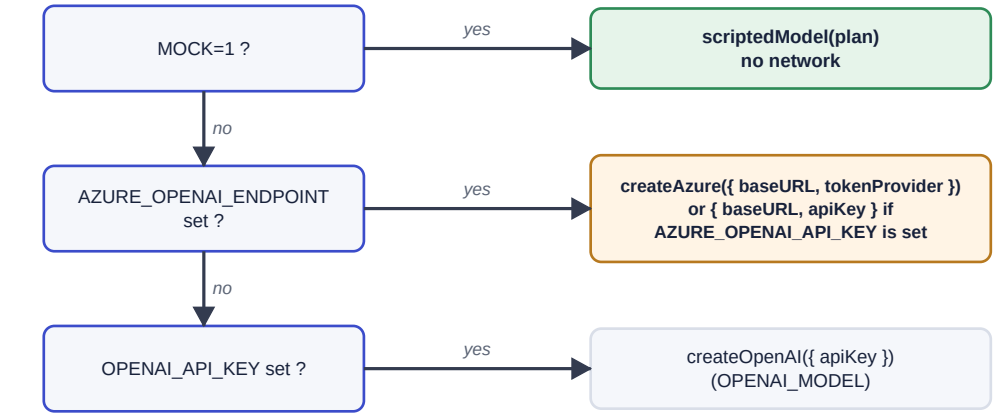
File	Change	Why
package.json, bun.lock	+ @ai-sdk/azure@^3, + @azure/identity	Azure provider for the AI SDK; Entra ID credential library.
labs/openai/_model.ts	rewrite of the real-model part (+ about 70 lines)	Single place that decides mock / azure / openai and builds the model. New: realModel(), azureTokenProvider(), azureBaseURL(), describeMode(), Azure error hints.
labs/openai/00-check-key.ts	rewrite (same size)	Two-stage check: first get an Entra token, then one tiny completion. Separates 'bad credentials' from 'missing role / wrong deployment'.
labs/openai/03-server.ts	5 lines	Was building its own OpenAI model; now calls realModel(). Health endpoint reports the mode.
labs/openai/04-decision-provider.ts	4 lines	Same: realModel() instead of createOpenAI.
labs/openai/01, 02, 03-client	none	They already go through getModel(plan) or make no model at all.
labs/openai/README.md	docs	New variables and provider order.
labs/openai/COPILOT_PROMPT_AZURE.md	new	The prompt from Part 5 as a file, so you can copy it without PDF line breaks.

IN SIMPLE WORDS Minimum change, in one sentence

Add two packages, teach one helper file to build an Azure model from a token provider, and make the three scripts that bypassed the helper use it. Everything else, including the gate, the engine, the rules and the mock mode, is byte-for-byte the same.

Part 3. The design of the change

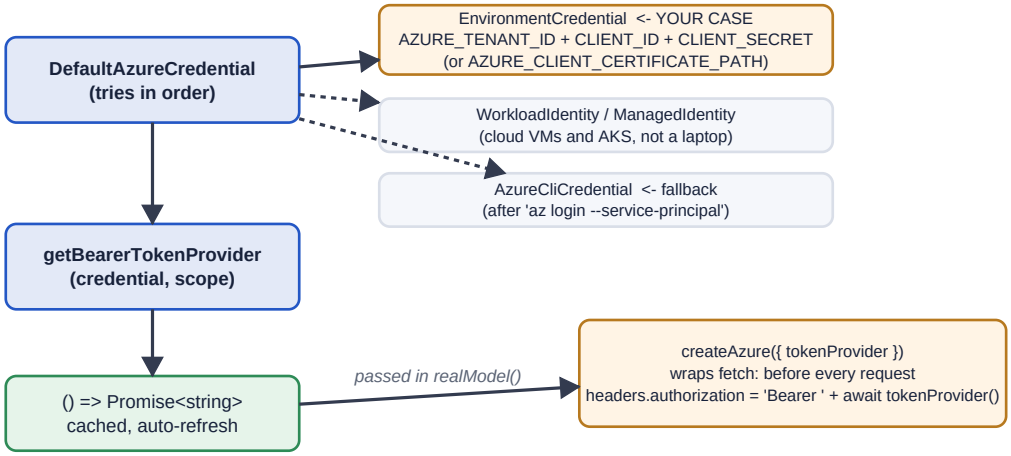
3.1 Provider selection from the environment



realModel() and getModel() follow this decision. No flags to pass; the variables you export decide.

> **WHY THIS DECISION** **Why environment variables and not a config file**
>
> Secrets must never be in files inside the repo. Env vars live in one shell window and vanish when it closes. It also keeps the lab commands identical to the OpenAI guide: only the `export` lines differ.

3.2 The token flow



Two library calls do all the OAuth work. Nothing in PetriFlow knows a token exists.

Two calls from @azure/identity plus one option of createAzure replace all OAuth plumbing.

> **WHY THIS DECISION** **Why DefaultAzureCredential and not ClientSecretCredential**
>
> ClientSecretCredential works only with a secret. DefaultAzureCredential tries a chain: the same env vars (secret *or* certificate), then cloud identities, then an existing `az login`. If your mentor hands you a certificate, or prefers `az login --service-principal`, the code does not change. Cost: a slower first failure when nothing is configured (it tries every link).

WHY THIS DECISION **Why @ai-sdk/azure and not the 'openai' npm package already in the repo**

The repo has the `openai` node SDK (used by one comparison script). Its AzureOpenAI client also accepts a token provider, but it is not a Vercel AI SDK LanguageModel, so the gate's wrapTools and generateText could not use it without an adapter. @ai-sdk/azure@3 is the provider that matches ai@6 and @ai-sdk/openai@3 already installed, and it has a `tokenProvider` option that is invoked on every request.

3.3 One request on the wire



What the laptop sends per model call once the token is in hand.

3.4 Endpoint normalisation

Admins hand out endpoints in different shapes. `azureBaseURL()` turns the bare resource URL into the v1 base the SDK expects, and leaves anything with a path alone so gateways keep working. Verified by running it:

azureBaseURL() outputs

```
https://foo.openai.azure.com          -> https://foo.openai.azure.com/openai/v1
https://foo.openai.azure.com/         -> https://foo.openai.azure.com/openai/v1
https://foo.openai.azure.com/openai/v1 -> https://foo.openai.azure.com/openai/v1
https://gw.corp.example/apim/aoai     -> https://gw.corp.example/apim/aoai
```

NEEDS RESEARCH **Things this guide could not verify without real credentials**

(1) Whether your resource is a classic Azure OpenAI resource (`*.openai.azure.com`) or a Foundry project endpoint (`*.services.ai.azure.com`); the token scope may differ for the latter, hence the AZURE_OPENAI_SCOPE override. (2) Whether an API gateway in front needs an extra subscription header; createAzure has a `headers` option for that. (3) Whether the newest @azure/identity honours AZURE_TOKEN_CREDENTIALS to shorten the credential chain. Report what you see on the first real run.

Part 4. The changes, file by file

Green lines are added, red lines removed. The full files are in Appendix B; the complete diff in Appendix A. Read this part once before letting Copilot touch anything, so you can judge its diff.

4.1 package.json: two dependencies

```
bun add @ai-sdk/azure@^3 @azure/identity
```

Observed output

```
installed @ai-sdk/azure@3.0.135
installed @azure/identity@4.13.3
35 packages installed [13.64s]
```

```
index 0942c7e..4579b9d 100644
--- a/package.json
+++ b/package.json
@@ -18,6 +18,9 @@
     "@types/bun": "^1"
   },
   "dependencies": {
+    "@ai-sdk/azure": "^3",
+    "@ai-sdk/openai": "^3",
+    "@azure/identity": "^4.13.3",
     "ai": "^6.0.86"
   }
 }
```

WHAT IS EXECUTING

Bun adds the lines to the root package.json and resolves 35 transitive packages (MSAL, the Azure core HTTP client). Nothing in packages/* imports them; only the labs do.

4.2 labs/openai/_model.ts: the plug

Six additions: a PROVIDER constant, MODEL_ID reading the deployment name in Azure mode, `azureTokenProvider()`, `azureBaseURL()`, `realModel()` and `describeMode()`; plus Azure branches in `explainError()`. The mock code is untouched.

```
--- a/labs/openai/_model.ts
+++ b/labs/openai/_model.ts
@@ -1,14 +1,29 @@
-// Shared helper for the OpenAI labs.
+// Shared helper for the LLM labs. Three ways to get a model, picked by env vars:
 //
-//   real mode : needs OPENAI_API_KEY. Model from OPENAI_MODEL (default gpt-4o-mini).
-//   mock mode : MOCK=1. A scripted fake model that returns the tool calls you give it.
-//               No network, no key, no cost. Use it to rehearse a lab before paying.
+//   mock mode  : MOCK=1. A scripted fake model that returns the tool calls you give it.
+//                No network, no key, no cost. Use it to rehearse a lab before paying.
+//   azure mode : AZURE_OPENAI_ENDPOINT is set. Auth is a service principal
+//                (AZURE_TENANT_ID / AZURE_CLIENT_ID / AZURE_CLIENT_SECRET) via Entra ID,
+//                or AZURE_OPENAI_API_KEY if you were given a key instead.
+//   openai mode: otherwise. Needs OPENAI_API_KEY. Model from OPENAI_MODEL.
 import { createOpenAI } from "@ai-sdk/openai";
+import { createAzure } from "@ai-sdk/azure";
+import { DefaultAzureCredential, getBearerTokenProvider } from "@azure/identity";
 import { MockLanguageModelV3 } from "ai/test";
 import type { LanguageModel } from "ai";
 export const MOCK = process.env.MOCK === "1";
-export const MODEL_ID = process.env.OPENAI_MODEL ?? "gpt-4o-mini";
+export const AZURE_ENDPOINT = process.env.AZURE_OPENAI_ENDPOINT;
+export const PROVIDER: "mock" | "azure" | "openai" =
+  MOCK ? "mock" : AZURE_ENDPOINT ? "azure" : "openai";
+
+// Azure: the "model" is the DEPLOYMENT NAME your admin created, not the model family.
+export const MODEL_ID = PROVIDER === "azure"
+  ? (process.env.AZURE_OPENAI_DEPLOYMENT ?? "")
+  : (process.env.OPENAI_MODEL ?? "gpt-4o-mini");
```

```
+
+// Token audience for Azure OpenAI. Rarely needs changing.
+const AZURE_SCOPE = process.env.AZURE_OPENAI_SCOPE ?? "https://cognitiveservices.azure.com/.def
   ault";
 export type PlanStep =
   | { tool: string; input?: Record<string, unknown> }
@@ -54,22 +69,60 @@
   }) as unknown as LanguageModel;
 }
-/** Real OpenAI model, or the scripted mock when MOCK=1. */
+function die(msg: string): never {
+  console.error(msg);
+  process.exit(1);
+}
+
+/** Entra ID token source. DefaultAzureCredential reads AZURE_TENANT_ID, AZURE_CLIENT_ID
+ *  and AZURE_CLIENT_SECRET (or AZURE_CLIENT_CERTIFICATE_PATH), and falls back to
+ *  `az login` if those are absent. Tokens are cached and refreshed automatically. */
+export function azureTokenProvider() {
+  return getBearerTokenProvider(new DefaultAzureCredential(), AZURE_SCOPE);
+}
+
+/** Normalise what the admin gave you into a base URL the AI SDK accepts.
+ *  "https://foo.openai.azure.com" -> "https://foo.openai.azure.com/openai/v1"
+ *  Anything with a path (gateway / APIM / already /openai/v1) is kept as-is. */
+export function azureBaseURL(endpoint: string): string {
+  const trimmed = endpoint.replace(/\/+$/, "");
+  const path = new URL(trimmed).pathname.replace(/\/+$/, "");
+  return path === "" ? `${trimmed}/openai/v1` : trimmed;
+}
+
+/** The real model for the current provider. Exits with a clear message if config is missing. *
   /
+export function realModel(): LanguageModel {
+  if (PROVIDER === "azure") {
+    if (!MODEL_ID) die("AZURE_OPENAI_DEPLOYMENT is not set (the deployment name, e.g. gpt-4o-mi
   ni).");
+    const apiKey = process.env.AZURE_OPENAI_API_KEY;
+    const azure = createAzure(apiKey
+      ? { baseURL: azureBaseURL(AZURE_ENDPOINT!), apiKey }
+      : { baseURL: azureBaseURL(AZURE_ENDPOINT!), tokenProvider: azureTokenProvider() });
+    return azure(MODEL_ID);
+  }
+  const key = process.env.OPENAI_API_KEY;
+  if (!key) die("OPENAI_API_KEY is not set. Export it, set AZURE_OPENAI_ENDPOINT, or run with M
   OCK=1.");
+  return createOpenAI({ apiKey: key })(MODEL_ID);
+}
+
+/** Real model, or the scripted mock when MOCK=1. */
 export function getModel(plan: PlanStep[]): LanguageModel {
   if (MOCK) return scriptedModel(plan);
-  const key = process.env.OPENAI_API_KEY;
-  if (!key) {
-    console.error("OPENAI_API_KEY is not set. Export it, or run with MOCK=1 to rehearse.");
-    process.exit(1);
-  }
-  return createOpenAI({ apiKey: key })(MODEL_ID);
+  return realModel();
+}
+
+export function describeMode(): string {
+  if (PROVIDER === "mock") return "MOCK (scripted model, no network)";
+  if (PROVIDER === "azure") {
+    const auth = process.env.AZURE_OPENAI_API_KEY ? "api key" : "service principal (Entra ID)";
+    return `REAL (Azure OpenAI, deployment ${MODEL_ID || "?"}, ${auth})`;
   }
+  return `REAL (OpenAI ${MODEL_ID})`;
 }
 export function banner(title: string) {
   console.log(`=== ${title} ===`);
-  console.log(MOCK
-    ? "mode: MOCK (scripted model, no network)"
-    : `mode: REAL (OpenAI ${MODEL_ID})`);
+  console.log(`mode: ${describeMode()}`);
   console.log("");
 }
@@ -99,18 +152,36 @@
   }
 }
-/** Friendly explanation of common OpenAI errors. */
+/** Friendly explanation of common OpenAI / Azure errors. */
 export function explainError(e: any) {
-  const status = e?.statusCode;
+  const status = e?.statusCode ?? e?.status;
```

```
   const msg = String(e?.message ?? e);
-  console.error("\nERROR:", msg.slice(0, 300));
-  if (status === 401) {
-    console.error("=> The key is wrong or revoked. Check OPENAI_API_KEY.");
+  console.error("\nERROR:", msg.slice(0, 400));
+  const aad = /AADSTS\d+/.exec(msg)?.[0];
+  if (aad || /CredentialUnavailable|EnvironmentCredential/i.test(msg)) {
+    console.error(`=> Entra ID login failed${aad ? ` (${aad})` : ""}.`);
+    console.error("   Check AZURE_TENANT_ID, AZURE_CLIENT_ID, AZURE_CLIENT_SECRET (all three),"
   );
+    console.error("   or run: az login --service-principal -u <clientId> -p <secret> --tenant <
   tenantId>");
+    if (aad === "AADSTS7000215") console.error("   AADSTS7000215 = the client secret is wrong o
  r expired.");
+    if (aad === "AADSTS700016") console.error("   AADSTS700016 = client id not found in that te
  nant.");
+    if (aad === "AADSTS90002") console.error("   AADSTS90002 = tenant id not found.");
+  } else if (status === 401 || status === 403) {
+    if (PROVIDER !== "azure") {
+      console.error("=> The key is wrong or revoked. Check OPENAI_API_KEY.");
+    } else if (process.env.AZURE_OPENAI_API_KEY) {
+      console.error("=> Azure rejected the key. Check AZURE_OPENAI_API_KEY and AZURE_OPENAI_END
  POINT.");
+    } else {
+      console.error("=> Entra issued a token but the resource refused it. The service principal
   needs\n"
+        + "   the role 'Cognitive Services OpenAI User' on the Azure OpenAI resource (ask the a
  dmin).");
+    }
  } else if (status === 429) {
-    console.error("=> Rate limit or no credit left (insufficient_quota). Check billing.");
+    console.error("=> Rate limit or quota exhausted. Wait, or ask for more TPM on the deploymen
  t.");
  } else if (status === 404) {
-    console.error(`=> Model '${MODEL_ID}' not found. Try OPENAI_MODEL=gpt-4o-mini.`);
-  } else if (/ENOTFOUND|ECONNRESET|ETIMEDOUT|fetch failed/i.test(msg)) {
-    console.error("=> Network. Set HTTPS_PROXY if your company uses a proxy.");
+    console.error(PROVIDER === "azure"
+      ? `=> Not found. Is AZURE_OPENAI_DEPLOYMENT='${MODEL_ID}' the exact deployment name,\n`
+        + `   and is AZURE_OPENAI_ENDPOINT the resource URL (${AZURE_ENDPOINT})?`
+      : `=> Model '${MODEL_ID}' not found. Try OPENAI_MODEL=gpt-4o-mini.`);
+  } else if (/ENOTFOUND|ECONNRESET|ETIMEDOUT|fetch failed|getaddrinfo/i.test(msg)) {
+    console.error("=> Network. Set HTTPS_PROXY if your company uses a proxy; check the endpoint
   host.");
  }
}
```

WHAT IS EXECUTING

At import time the file reads MOCK and AZURE_OPENAI_ENDPOINT once. `realModel()` builds a DefaultAzureCredential, wraps it with getBearerTokenProvider for the Cognitive Services scope, and hands that function to createAzure. No network happens yet: the first token is fetched lazily when the first request goes out, and refreshed automatically before it expires.

4.3 labs/openai/00-check-key.ts: a two-stage check

```
--- a/labs/openai/00-check-key.ts
+++ b/labs/openai/00-check-key.ts
@@ -1,25 +1,57 @@
-// Step 0: prove the key works and see how many tokens a tiny call costs.
-//   OPENAI_API_KEY=sk-... bun run labs/openai/00-check-key.ts
+// Step 0: prove the credentials work and see how many tokens a tiny call costs.
+//   OpenAI : OPENAI_API_KEY=sk-... bun run labs/openai/00-check-key.ts
+//   Azure  : export AZURE_OPENAI_ENDPOINT, AZURE_OPENAI_DEPLOYMENT,
+//            AZURE_TENANT_ID, AZURE_CLIENT_ID, AZURE_CLIENT_SECRET
+//            bun run labs/openai/00-check-key.ts
 import { generateText } from "ai";
-import { createOpenAI } from "@ai-sdk/openai";
-import { MODEL_ID, explainError } from "./_model";
+import {
+  PROVIDER, MODEL_ID, AZURE_ENDPOINT, azureBaseURL, azureTokenProvider,
+  describeMode, realModel, explainError,
+} from "./_model";
-const key = process.env.OPENAI_API_KEY;
-if (!key) {
-  console.error("OPENAI_API_KEY is not set. In Git Bash:  export OPENAI_API_KEY=sk-...");
+function mask(v: string | undefined) {
+  return v ? `${v.slice(0, 4)}...${v.slice(-4)}` : "(not set)";
+}
```

```
+
+console.log(`mode: ${describeMode()}`);
+if (PROVIDER === "azure") {
+  console.log(`endpoint  : ${AZURE_ENDPOINT}  ->  ${azureBaseURL(AZURE_ENDPOINT!)}`);
+  console.log(`deployment: ${MODEL_ID || "(not set)"}`);
+  console.log(`tenant    : ${mask(process.env.AZURE_TENANT_ID)}`);
+  console.log(`client id : ${mask(process.env.AZURE_CLIENT_ID)}`);
+  console.log(`secret    : ${process.env.AZURE_CLIENT_SECRET ? "(set)" : "(not set)"}`);
+  if (!MODEL_ID) {
+    console.error("AZURE_OPENAI_DEPLOYMENT is not set. Ask the admin for the deployment name.")
    ;
+    process.exit(1);
+  }
+  if (!process.env.AZURE_OPENAI_API_KEY) {
+    // Step A: get a token from Entra ID. This proves tenant/client/secret.
+    try {
+      const t0 = Date.now();
+      const token = await azureTokenProvider()();
+      console.log(`entra token: OK, ${token.length} chars (${Date.now() - t0} ms)`);
+    } catch (e) {
+      explainError(e);
+      process.exit(1);
+    }
+  }
+} else if (PROVIDER === "openai") {
+  console.log(`key: ${mask(process.env.OPENAI_API_KEY)}  model: ${MODEL_ID}`);
+} else {
+  console.log("MOCK=1 makes no sense here: this script exists to test real credentials.");
   process.exit(1);
 }
-console.log(`key: ${key.slice(0, 7)}...${key.slice(-4)}  model: ${MODEL_ID}`);
+// Step B: one tiny completion. This proves the role on the resource and the deployment name.
 try {
   const t0 = Date.now();
   const r = await generateText({
-    model: createOpenAI({ apiKey: key })(MODEL_ID),
+    model: realModel(),
     prompt: "Reply with exactly the two letters: OK",
   });
   console.log(`reply: ${JSON.stringify(r.text)}  (${Date.now() - t0} ms)`);
   console.log(`tokens: input=${r.usage.inputTokens} output=${r.usage.outputTokens}`);
-  console.log("Key works. You can run the other labs without MOCK=1.");
+  console.log("Credentials work. You can run the other labs without MOCK=1.");
 } catch (e) {
   explainError(e);
   process.exit(1);
```

WHAT IS EXECUTING

Stage A calls the token provider directly. If it fails you have an identity problem (tenant, client id, secret) and Azure OpenAI was never contacted. Stage B makes one completion. If A passes and B fails with 401, the identity is fine and the role is missing; 404 means the deployment name or endpoint is wrong. This split is what you report to your mentor.

4.4 labs/openai/03-server.ts: five lines

```
--- a/labs/openai/03-server.ts
+++ b/labs/openai/03-server.ts
@@ -1,14 +1,13 @@
 // Lab 3: a localhost chat server. The gate is stateless; state comes from message history.
 //   rehearse:  MOCK=1 bun run labs/openai/03-server.ts
-//   real:      OPENAI_API_KEY=sk-... bun run labs/openai/03-server.ts
+//   real:      OPENAI_API_KEY=sk-... (or the AZURE_* vars) bun run labs/openai/03-server.ts
 // Then, in a second Git Bash window:  bun run labs/openai/03-client.ts
 import { generateText, tool, stepCountIs } from "ai";
 import type { ModelMessage } from "ai";
 import { z } from "zod";
 import { compile } from "@petriflow/rules";
 import { createPetriflowGate } from "@petriflow/vercel-ai";
-import { createOpenAI } from "@ai-sdk/openai";
-import { MOCK, MODEL_ID, scriptedModel, banner } from "./_model";
+import { MOCK, scriptedModel, realModel, describeMode, banner } from "./_model";
 banner("Lab 3: localhost server with replay");
 const PORT = Number(process.env.PORT ?? 3456);
@@ -43,7 +42,7 @@
 // 3. Pick the model. The mock reacts to the last user message.
 function modelFor(prompt: string) {
-  if (!MOCK) return createOpenAI({ apiKey: process.env.OPENAI_API_KEY ?? "" })(MODEL_ID);
+  if (!MOCK) return realModel();
```

```
   const p = prompt.toLowerCase();
   if (p.includes("test")) {
     return scriptedModel([{ tool: "test" }, { text: "Tests passed." }]);
@@ -57,10 +56,7 @@
   return scriptedModel([{ text: "I can run tests or deploy." }]);
 }
-if (!MOCK && !process.env.OPENAI_API_KEY) {
-  console.error("OPENAI_API_KEY is not set. Export it, or run with MOCK=1.");
-  process.exit(1);
-}
+if (!MOCK) realModel(); // fail fast if the provider config is missing
 const SYSTEM_PREFIX = "You are a release assistant. Use the tools. "
   + "If a tool is blocked, explain the reason.\n\n";
@@ -71,7 +67,7 @@
   async fetch(req) {
     const url = new URL(req.url);
     if (req.method === "GET" && url.pathname === "/") {
-      const mode = MOCK ? "mock" : `openai:${MODEL_ID}`;
+      const mode = describeMode();
       return Response.json({ ok: true, rules: verification, mode });
     }
     if (req.method === "POST" && url.pathname === "/chat") {
```

4.5 labs/openai/04-decision-provider.ts: four lines

```
--- a/labs/openai/04-decision-provider.ts
+++ b/labs/openai/04-decision-provider.ts
@@ -1,14 +1,13 @@
 // Lab 4: the engine side. OpenAI chooses which transition fires at a choice point.
 //   rehearse:  MOCK=1 bun run labs/openai/04-decision-provider.ts
-//   real:      OPENAI_API_KEY=sk-... bun run labs/openai/04-decision-provider.ts
+//   real:      OPENAI_API_KEY=sk-... (or the AZURE_* vars) bun run labs/openai/04-decision-pro
   vider.ts
 import { generateText, tool } from "ai";
 import { z } from "zod";
 import { Database } from "bun:sqlite";
 import { Scheduler, createExecutor, sqliteAdapter } from "@petriflow/engine";
 import type { DecisionProvider } from "@petriflow/engine/decision";
-import { createOpenAI } from "@ai-sdk/openai";
 import { definition } from "../../workflows/simple-agent/definition";
-import { MOCK, MODEL_ID, banner, explainError } from "./_model";
+import { MOCK, realModel, banner, explainError } from "./_model";
 banner("Lab 4: OpenAI as the engine's decision provider");
@@ -37,7 +36,7 @@
       .map((t) => `${t.name} [${t.inputs}] -> [${t.outputs}]`)
       .join("; ");
     const result = await generateText({
-      model: createOpenAI({ apiKey: process.env.OPENAI_API_KEY ?? "" })(MODEL_ID),
+      model: realModel(),
       system: `You control the workflow "${req.workflowName}". `
         + "Prefer to iterate once to gather more tool results, then generate. "
         + "Pick exactly one transition.",
@@ -56,10 +55,7 @@
   },
 };
-if (!MOCK && !process.env.OPENAI_API_KEY) {
-  console.error("OPENAI_API_KEY is not set. Export it, or run with MOCK=1.");
-  process.exit(1);
-}
+if (!MOCK) realModel(); // fail fast if the provider config is missing
 const db = new Database(":memory:");
 const executor = createExecutor(definition as any, {
```

> **WHAT IS EXECUTING**
>
> Both scripts had copied the model construction instead of using the helper. Now they call `realModel()`, and the start-up guard becomes `if (!MOCK) realModel()`, which exits early with a clear message if configuration is missing, for either provider.

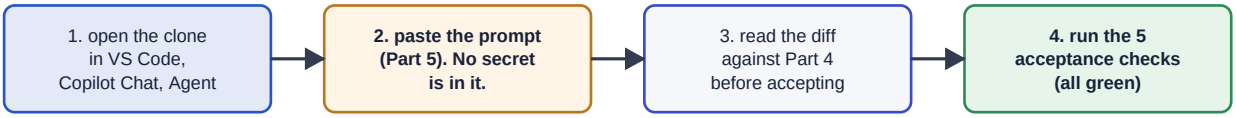
4.6 Optional: the repo's own examples on Azure

Not required for the labs. If you ever want `examples/01..04/agent.ts` to use Azure, it is the same two lines in each file. Verified to typecheck on example 02; the examples still have the Windows .rules-path issue described in the earlier guides.

```
for f in examples/0*/agent.ts; do
  sed -i -e 's|^import { anthropic } from "@ai-sdk/anthropic";|import { realModel } from "../../labs/op
         -e 's|model: anthropic("claude-sonnet-4-5-20250929"),|model: realModel(),|' "$f"
done
git diff --stat examples/
```

Revert with `git checkout -- examples/`.

Part 5. The GitHub Copilot prompt



If Copilot drifts (renames things, touches packages/), reject and paste the Appendix B files instead. Same end state, zero AI involved.

Never paste the client secret into Copilot Chat: the chat is sent to a cloud service.

The safe loop: prompt, review, verify, fall back to the Appendix if needed.

5.1 Before you paste

- Open the company clone of petri-flow in VS Code. Make sure `bun run build` has been done once (the labs import @petriflow/* from the built packages).
- If `labs/openai/` does not exist yet on that machine, create it from the previous guide's Appendix first. The prompt modifies existing files.
- Use Copilot Chat in **Agent** mode (or Edits mode) so it can edit several files and run the checks. Ask for the diff first; the prompt says so.
- The prompt contains no secret and needs none. Keep it that way.

5.2 The prompt (also in labs/openai/COPILOT_PROMPT_AZURE.md)

```
You are working in the petri-flow repository: a Bun + TypeScript monorepo. Goal: let the
scripts in `labs/openai/` talk to **Azure OpenAI authenticated with a Microsoft Entra ID
service principal**, with the smallest possible change. Do NOT modify anything under
`packages/`, `workflows/`, `apps/`, `site/`, `comparisons/` or `examples/`. The PetriFlow
gate and engine are provider-agnostic; only the place where the model object is built changes.
## Context you can rely on
- The labs use the Vercel AI SDK (`ai@6`). A model is a `LanguageModel` object; the gate never
  sees it. `labs/openai/_model.ts` builds the model in `getModel(plan)`. Three scripts still
  call `createOpenAI` directly: `00-check-key.ts`, `03-server.ts`, `04-decision-provider.ts`.
- `@ai-sdk/azure@3` exports `createAzure(options)`. Relevant options: `baseURL`, `apiKey`,
  `tokenProvider: () => Promise<string>` (called on every request; the SDK adds
  `Authorization: Bearer <token>`). `apiKey` and `tokenProvider` must not both be set.
  A base URL ending in `/openai/v1` is used as-is. `createAzure(...)("<deployment>")` returns
  the model; the id is the Azure **deployment name**, not the model family.
- `@azure/identity` exports `DefaultAzureCredential` (reads `AZURE_TENANT_ID`,
  `AZURE_CLIENT_ID`, `AZURE_CLIENT_SECRET` or `AZURE_CLIENT_CERTIFICATE_PATH`, and falls back
  to `az login`) and `getBearerTokenProvider(credential, scope)` which returns a cached,
  auto-refreshing `() => Promise<string>`. Scope: `https://cognitiveservices.azure.com/.default`.
## Steps
1. At the repo root run: `bun add @ai-sdk/azure@^3 @azure/identity`
2. Edit `labs/openai/_model.ts`:
   a. Add imports: `createAzure` from `@ai-sdk/azure`; `DefaultAzureCredential` and
      `getBearerTokenProvider` from `@azure/identity`.
   b. `export const AZURE_ENDPOINT = process.env.AZURE_OPENAI_ENDPOINT;`
      `export const PROVIDER: "mock" | "azure" | "openai" =
        MOCK ? "mock" : AZURE_ENDPOINT ? "azure" : "openai";`
   c. `MODEL_ID`: in azure mode use `process.env.AZURE_OPENAI_DEPLOYMENT ?? ""`, otherwise keep
      `process.env.OPENAI_MODEL ?? "gpt-4o-mini"`.
   d. `export function azureTokenProvider()` returning
      `getBearerTokenProvider(new DefaultAzureCredential(), process.env.AZURE_OPENAI_SCOPE ??
      "https://cognitiveservices.azure.com/.default")`.
   e. `export function azureBaseURL(endpoint: string): string`: strip trailing slashes; if the
      URL has an empty path, append `/openai/v1`; otherwise return it unchanged (gateways and
      full `/openai/v1` URLs pass through).
   f. `export function realModel(): LanguageModel`:
      - azure: if `MODEL_ID` is empty, print "AZURE_OPENAI_DEPLOYMENT is not set" and exit(1).
```

```
        If `AZURE_OPENAI_API_KEY` is set use `createAzure({ baseURL, apiKey })`, else
        `createAzure({ baseURL, tokenProvider: azureTokenProvider() })`. Return `azure(MODEL_ID)`.
      - openai: unchanged behaviour (exit with a message if `OPENAI_API_KEY` is missing;
        mention `AZURE_OPENAI_ENDPOINT` and `MOCK=1` as alternatives).
   g. `getModel(plan)` becomes `MOCK ? scriptedModel(plan) : realModel()`.
   h. `export function describeMode(): string` returning one line, e.g.
      "MOCK (scripted model, no network)",
      "REAL (Azure OpenAI, deployment <name>, service principal (Entra ID))",
      "REAL (OpenAI gpt-4o-mini)". `banner()` prints `mode: ${describeMode()}`.
   i. `explainError`: read status from `e.statusCode ?? e.status`. Add branches:
      - message contains `AADSTS\d+` or `CredentialUnavailable`/`EnvironmentCredential`:
        say Entra login failed, name the AADSTS code, tell the user to check the three
        AZURE_* variables or run `az login --service-principal`. Special-case AADSTS7000215
        (bad secret), AADSTS700016 (client id not in tenant), AADSTS90002 (tenant not found).
      - 401/403 in azure mode: with `AZURE_OPENAI_API_KEY` say the key or endpoint is wrong;
        otherwise say the principal needs the role "Cognitive Services OpenAI User" on the
        resource.
      - 404 in azure mode: say to check `AZURE_OPENAI_DEPLOYMENT` (exact deployment name) and
        `AZURE_OPENAI_ENDPOINT`.
      - keep the existing OpenAI branches and the network branch (add `getaddrinfo`).
3. Rewrite `labs/openai/00-check-key.ts`: print `describeMode()`; in azure mode print the
   endpoint, the normalised base URL, the deployment, masked tenant and client ids (first 4 and
   last 4 characters) and whether the secret is set, never its value. Exit if the deployment is
   missing. In service-principal mode first call `await azureTokenProvider()()` and print
   "entra token: OK" with the token length and elapsed ms (on failure: `explainError`, exit 1).
   Then one `generateText({ model: realModel(), prompt: "Reply with exactly the two letters: OK" })`
   and print reply, elapsed ms and `r.usage.inputTokens` / `outputTokens`.
   In openai mode keep the old behaviour. With `MOCK=1` print a note and exit 1.
4. Edit `labs/openai/03-server.ts` and `labs/openai/04-decision-provider.ts`:
   - delete the `createOpenAI` import; import `realModel` (and `describeMode` in 03) from
     `./_model` instead of `MODEL_ID`;
   - replace `createOpenAI({ apiKey: process.env.OPENAI_API_KEY ?? "" })(MODEL_ID)` with
     `realModel()`;
   - replace the `if (!MOCK && !process.env.OPENAI_API_KEY) {...exit...}` guard with
     `if (!MOCK) realModel(); // fail fast if the provider config is missing`;
   - in 03, the GET / health response uses `mode: describeMode()`.
5. Do not touch `01-file-agent.ts`, `02-deploy-agent.ts`, `03-client.ts`.
6. Update `labs/openai/README.md`: provider order (MOCK, then AZURE_OPENAI_ENDPOINT, then
   OpenAI) and the Azure variables: `AZURE_OPENAI_ENDPOINT`, `AZURE_OPENAI_DEPLOYMENT`,
   `AZURE_TENANT_ID`, `AZURE_CLIENT_ID`, `AZURE_CLIENT_SECRET`, optional
   `AZURE_OPENAI_API_KEY`, optional `AZURE_OPENAI_SCOPE`; note the required role.
## Constraints
- Keep every line under 100 characters. Keep `MOCK=1` behaviour identical.
- Never print, log or write a secret. No new files. No changes outside `labs/openai/`,
  `package.json` and `bun.lock`.
- Show me the full diff before applying it.
## Acceptance (run all of these; all must pass)
```bash
bunx tsc --noEmit --strict --skipLibCheck --target es2022 --module esnext \
  --moduleResolution bundler --types bun --allowImportingTsExtensions labs/openai/*.ts
MOCK=1 bun run labs/openai/01-file-agent.ts                # ends with "blocked calls: 2"
MOCK=1 AUTO_APPROVE=1 bun run labs/openai/02-deploy-agent.ts
MOCK=1 bun run labs/openai/04-decision-provider.ts         # ends with "status":"completed"
# wiring test with FAKE credentials: must fail with AADSTS90002 and the friendly explanation
AZURE_OPENAI_ENDPOINT=https://x.openai.azure.com AZURE_OPENAI_DEPLOYMENT=d \
AZURE_TENANT_ID=11111111-2222-3333-4444-555555555555 \
AZURE_CLIENT_ID=66666666-7777-8888-9999-000000000000 AZURE_CLIENT_SECRET=x \
bun run labs/openai/00-check-key.ts
```
```

Copy from the file rather than from this PDF if your reader breaks long lines.

5.3 What to check in Copilot's diff

Check	Pass looks like	Fail looks like
Scope	Only labs/openai/*, package.json, bun.lock	Anything under packages/, workflows/, examples/
Mock path	scriptedModel() untouched; getModel is MOCK ? scripted : realModel()	New branches inside the mock, renamed exports

Check	Pass looks like	Fail looks like
Auth	createAzure({ baseURL, tokenProvider }) with getBearerTokenProvider(new DefaultAzureCredential(), scope)	apiKey: process.env.AZURE_CLIENT_SECRET, or both apiKey and tokenProvider
Deployment	MODEL_ID from AZURE_OPENAI_DEPLOYMENT in azure mode	Hard-coded 'gpt-4o-mini' sent to Azure
Secrets	Masked ids, secret shown as (set)	console.log of the secret or the token
Acceptance	All five commands pass, the fake-credential run prints AADSTS90002	Typecheck errors, or the fake run 'succeeds'

IN SIMPLE WORDS **Plan B**

Reject Copilot's edit and paste the five blocks of Appendix B into Git Bash. That recreates exactly the files verified for this guide.

Part 6. Run it

Step 6.1 Export the environment (one Git Bash window)

```
export AZURE_OPENAI_ENDPOINT=https://<resource>.openai.azure.com
export AZURE_OPENAI_DEPLOYMENT=<deployment-name>
export AZURE_TENANT_ID=<tenant guid>
export AZURE_CLIENT_ID=<client guid>
export AZURE_CLIENT_SECRET='<secret value>'   # keep the quotes
env | grep -c '^AZURE_'                       # expect 5
echo ${AZURE_CLIENT_SECRET:0:3}...            # first 3 chars only
```

Alternative if the Azure CLI is available and allowed: `az login --service-principal -u $AZURE_CLIENT_ID -p $AZURE_CLIENT_SECRET --tenant $AZURE_TENANT_ID`, then the three AZURE_TENANT/CLIENT variables are not needed; DefaultAzureCredential finds the CLI login.

Step 6.2 Lab 0 with FAKE values first (proves the wiring, costs nothing)

```
AZURE_OPENAI_ENDPOINT=https://x.openai.azure.com AZURE_OPENAI_DEPLOYMENT=d \
AZURE_TENANT_ID=11111111-2222-3333-4444-555555555555 \
AZURE_CLIENT_ID=66666666-7777-8888-9999-000000000000 AZURE_CLIENT_SECRET=x \
bun run labs/openai/00-check-key.ts
```

Observed output with fake values (this is the GOOD result)

```
mode: REAL (Azure OpenAI, deployment d, service principal (Entra ID))
endpoint  : https://x.openai.azure.com  ->  https://x.openai.azure.com/openai/v1
deployment: d
tenant    : 1111...5555
client id : 6666...0000
secret    : (set)
ERROR: EnvironmentCredential authentication failed. To troubleshoot, visit https://aka.ms/az
    sdk/js/identity/environmentcredential/troubleshoot. Status code: 400
More details:
invalid_request: Error(s): 90002 - Timestamp: 2026-10-09 06:47:11Z - Description: AADSTS9000
    2: Tenant '11111111-2222-3333-4444-555555555555' not found. ...
=> Entra ID login failed (AADSTS90002).
   Check AZURE_TENANT_ID, AZURE_CLIENT_ID, AZURE_CLIENT_SECRET (all three),
   or run: az login --service-principal -u <clientId> -p <secret> --tenant <tenantId>
   AADSTS90002 = tenant id not found.
```

WHAT IS EXECUTING

The script reached login.microsoftonline.com, which answered that the tenant does not exist. That proves: the packages are installed, the code path is wired, and the laptop can reach Entra ID through the proxy. A 'fetch failed' or timeout here means the network blocks Microsoft login; set HTTPS_PROXY.

Step 6.3 Lab 0 with the real values

```
bun run labs/openai/00-check-key.ts
```

EXPECTED output (not observed: no real credentials on the authoring machine)

```
mode: REAL (Azure OpenAI, deployment gpt-4o-mini, service principal (Entra ID))
endpoint  : https://acme-aoai.openai.azure.com  ->  https://acme-aoai.openai.azure.com/opena
    i/v1
deployment: gpt-4o-mini
tenant    : 72f9...db47
client id : 1a2b...9f0e
secret    : (set)
entra token: OK, 1500 chars (650 ms)
reply: "OK"  (900 ms)
tokens: input=14 output=1
Credentials work. You can run the other labs without MOCK=1.
```

WHAT IS EXECUTING

Stage A exchanges id + secret for a token at Entra ID. Stage B sends one chat completion to the deployment with that token as a Bearer header. The usage line is what Azure metered. If stage A passes and stage B gives 401, ask for the role; 404, check the deployment name.

Step 6.4 The four labs, unchanged commands

```
bun run labs/openai/01-file-agent.ts
bun run labs/openai/02-deploy-agent.ts            # type y or n at each deploy
bun run labs/openai/03-server.ts                  # window 1, export AZURE_* too
bun run labs/openai/03-client.ts                  # window 2
bun run labs/openai/04-decision-provider.ts
```

Expected behaviour is the same as in the OpenAI guide, Part 5: the banner line reads `mode: REAL (Azure OpenAI, deployment ..., service principal (Entra ID))`, and the health endpoint of lab 3 returns that string in its `mode` field. The model's exact wording will differ; the gate's blocks and approvals will not.

Step 6.5 Rehearsal is still free

```
MOCK=1 bun run labs/openai/01-file-agent.ts
MOCK=1 AUTO_APPROVE=1 bun run labs/openai/02-deploy-agent.ts
MOCK=1 bun run labs/openai/04-decision-provider.ts
```

Observed after the change: identical to before

```
=== Lab 1: file cleanup agent ===
mode: MOCK (scripted model, no network)
...
blocked calls: 2
  delete: delete requires a successful call to backup first.
  rm: rm is blocked and cannot be called.
final net state: require-backup-before-delete: ready:1 | block-rm: ready:1
```

Part 7. Troubleshooting, hygiene, verification, cleanup

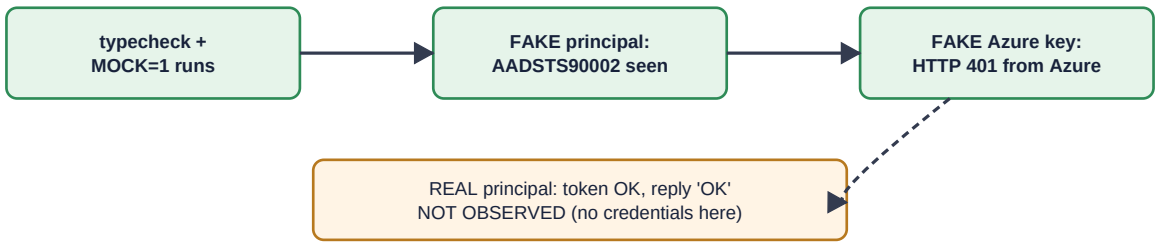
7.1 Error to cause to fix

You see	Stage	Meaning	Fix
AADSTS90002 Tenant not found	A	Wrong AZURE_TENANT_ID	Copy the tenant GUID again; it is not the subscription id.
AADSTS700016 Application not found	A	Client id not in that tenant	Check AZURE_CLIENT_ID (the Application (client) ID).
AADSTS7000215 Invalid client secret	A	Secret wrong, pasted the secret ID, or expired	Use the secret VALUE; ask for a new one if expired.
EnvironmentCredential is unavailable / CredentialUnavailableError	A	One of the three variables missing	env | grep AZURE_ ; export the missing one.
fetch failed / ENOTFOUND login.microsoftonline.com	A	Proxy or firewall	export HTTPS_PROXY=http://proxy:port ; ask IT.
401 PermissionDenied (after token OK)	B	Principal has no role on the resource	Admin assigns 'Cognitive Services OpenAI User' on the resource (or resource group).
401 Access denied due to invalid subscription key	B	You are in api-key mode with a bad key	Unset AZURE_OPENAI_API_KEY to use the principal, or fix the key.
404 DeploymentNotFound / Resource not found	B	Deployment name or endpoint wrong	Exact deployment name from the admin; endpoint without /openai/deployments/... suffix.
429 rate limit	B	Deployment TPM quota exhausted	Wait; ask for more tokens-per-minute on the deployment.
Both apiKey and tokenProvider were provided	startup	AZURE_OPENAI_API_KEY and principal both configured in a modified file	The verified realModel() never passes both; compare with Appendix B.
Cannot find module @ai-sdk/azure	startup	bun add not run on this machine	bun add @ai-sdk/azure@^3 @azure/identity at the repo root.

7.2 Secret hygiene on a company laptop

- The secret is in your shell only. Close the window when done, or `unset AZURE_CLIENT_SECRET`.
- Never paste it into Copilot Chat, a browser AI, a ticket, or a commit. The Copilot prompt in Part 5 needs none of the values.
- `git status --short` before any commit; labs/openai/ should contain only .ts and .md files.
- Note the secret's expiry date from your mentor. Stage A will start failing with AADSTS7000222 or 7000215 on that day.
- If the laptop is shared or imaged, ask whether a certificate (AZURE_CLIENT_CERTIFICATE_PATH) is preferred; the code already supports it.

7.3 What was verified, and what was not



Green boxes were run on 9 October 2026 on this machine. The orange box is what you will see first; report it back.

Verification status on the authoring machine, 9 October 2026.

- **Verified:** typecheck of all lab files; labs 1, 2, 3 (server + client) and 4 in MOCK=1 mode after the change, identical output; URL normalisation; missing-deployment exit; fake service principal reaches Entra ID and fails with AADSTS90002 with the friendly explanation; fake Azure API key reaches Azure OpenAI and gets HTTP 401; the optional example swap typechecks.
- **Not verified:** any call with real company credentials. The 'expected' block in 6.3 is derived from the code and the SDK, not observed. The first real run is yours; if its output differs, the Part 7.1 table is the map.

7.4 Cleanup

```
unset AZURE_CLIENT_SECRET AZURE_CLIENT_ID AZURE_TENANT_ID  # or close window
rm -rf /c/tmp/petriflow-lab
git checkout -- package.json bun.lock   # drops the 3 dependency lines
git status --short                      # labs/ is untracked: keep or delete
```

Appendix A. The complete diff

Old OpenAI-only lab files on the left, Azure-capable files on the right. Generated with `diff -u` on 9 October 2026.

```
--- a/labs/openai/_model.ts
+++ b/labs/openai/_model.ts
@@ -1,14 +1,29 @@
-// Shared helper for the OpenAI labs.
+// Shared helper for the LLM labs. Three ways to get a model, picked by env vars:
 //
-//   real mode : needs OPENAI_API_KEY. Model from OPENAI_MODEL (default gpt-4o-mini).
-//   mock mode : MOCK=1. A scripted fake model that returns the tool calls you give it.
-//               No network, no key, no cost. Use it to rehearse a lab before paying.
+//   mock mode  : MOCK=1. A scripted fake model that returns the tool calls you give it.
+//                No network, no key, no cost. Use it to rehearse a lab before paying.
+//   azure mode : AZURE_OPENAI_ENDPOINT is set. Auth is a service principal
+//                (AZURE_TENANT_ID / AZURE_CLIENT_ID / AZURE_CLIENT_SECRET) via Entra ID,
+//                or AZURE_OPENAI_API_KEY if you were given a key instead.
+//   openai mode: otherwise. Needs OPENAI_API_KEY. Model from OPENAI_MODEL.
 import { createOpenAI } from "@ai-sdk/openai";
+import { createAzure } from "@ai-sdk/azure";
+import { DefaultAzureCredential, getBearerTokenProvider } from "@azure/identity";
 import { MockLanguageModelV3 } from "ai/test";
 import type { LanguageModel } from "ai";
 export const MOCK = process.env.MOCK === "1";
-export const MODEL_ID = process.env.OPENAI_MODEL ?? "gpt-4o-mini";
+export const AZURE_ENDPOINT = process.env.AZURE_OPENAI_ENDPOINT;
+export const PROVIDER: "mock" | "azure" | "openai" =
+  MOCK ? "mock" : AZURE_ENDPOINT ? "azure" : "openai";
+
+// Azure: the "model" is the DEPLOYMENT NAME your admin created, not the model family.
+export const MODEL_ID = PROVIDER === "azure"
+  ? (process.env.AZURE_OPENAI_DEPLOYMENT ?? "")
+  : (process.env.OPENAI_MODEL ?? "gpt-4o-mini");
+
+// Token audience for Azure OpenAI. Rarely needs changing.
+const AZURE_SCOPE = process.env.AZURE_OPENAI_SCOPE ?? "https://cognitiveservices.azure.com/.default";
 export type PlanStep =
   | { tool: string; input?: Record<string, unknown> }
@@ -54,22 +69,60 @@
   }) as unknown as LanguageModel;
 }
-/** Real OpenAI model, or the scripted mock when MOCK=1. */
+function die(msg: string): never {
+  console.error(msg);
+  process.exit(1);
+}
+
+/** Entra ID token source. DefaultAzureCredential reads AZURE_TENANT_ID, AZURE_CLIENT_ID
+ *  and AZURE_CLIENT_SECRET (or AZURE_CLIENT_CERTIFICATE_PATH), and falls back to
+ *  `az login` if those are absent. Tokens are cached and refreshed automatically. */
+export function azureTokenProvider() {
+  return getBearerTokenProvider(new DefaultAzureCredential(), AZURE_SCOPE);
+}
+
+/** Normalise what the admin gave you into a base URL the AI SDK accepts.
+ *  "https://foo.openai.azure.com" -> "https://foo.openai.azure.com/openai/v1"
+ *  Anything with a path (gateway / APIM / already /openai/v1) is kept as-is. */
+export function azureBaseURL(endpoint: string): string {
+  const trimmed = endpoint.replace(/\/+$/, "");
+  const path = new URL(trimmed).pathname.replace(/\/+$/, "");
+  return path === "" ? `${trimmed}/openai/v1` : trimmed;
+}
+
+/** The real model for the current provider. Exits with a clear message if config is missing. */
 export function realModel(): LanguageModel {
-  const key = process.env.OPENAI_API_KEY;
-  if (!key) die("OPENAI_API_KEY is not set. Export it, or run with MOCK=1 for the scripted mock.");
-  return createOpenAI({ apiKey: key })(MODEL_ID);
+  if (PROVIDER === "azure") {
+    if (!MODEL_ID) die("AZURE_OPENAI_DEPLOYMENT is not set (the deployment name, e.g. gpt-4o-mini).");
+    const apiKey = process.env.AZURE_OPENAI_API_KEY;
+    const azure = createAzure(apiKey
+      ? { baseURL: azureBaseURL(AZURE_ENDPOINT!), apiKey }
+      : { baseURL: azureBaseURL(AZURE_ENDPOINT!), tokenProvider: azureTokenProvider() });
+    return azure(MODEL_ID);
+  }
+  const key = process.env.OPENAI_API_KEY;
+  if (!key) die("OPENAI_API_KEY is not set. Export it, set AZURE_OPENAI_ENDPOINT, or run with M
```

```
   OCK=1.");
+  return createOpenAI({ apiKey: key })(MODEL_ID);
+}
+
+/** Real model, or the scripted mock when MOCK=1. */
 export function getModel(plan: PlanStep[]): LanguageModel {
   if (MOCK) return scriptedModel(plan);
-  const key = process.env.OPENAI_API_KEY;
-  if (!key) {
-    console.error("OPENAI_API_KEY is not set. Export it, or run with MOCK=1 to rehearse.");
-    process.exit(1);
+  return realModel();
+}
+
+export function describeMode(): string {
+  if (PROVIDER === "mock") return "MOCK (scripted model, no network)";
+  if (PROVIDER === "azure") {
+    const auth = process.env.AZURE_OPENAI_API_KEY ? "api key" : "service principal (Entra ID)";
+    return `REAL (Azure OpenAI, deployment ${MODEL_ID || "?"}, ${auth})`;
   }
-  return createOpenAI({ apiKey: key })(MODEL_ID);
+  return `REAL (OpenAI ${MODEL_ID})`;
 }
 export function banner(title: string) {
   console.log(`=== ${title} ===`);
-  console.log(MOCK
-    ? "mode: MOCK (scripted model, no network)"
-    : `mode: REAL (OpenAI ${MODEL_ID})`);
+  console.log(`mode: ${describeMode()}`);
   console.log("");
 }
@@ -99,18 +152,36 @@
   }
 }
-/** Friendly explanation of common OpenAI errors. */
+/** Friendly explanation of common OpenAI / Azure errors. */
 export function explainError(e: any) {
-  const status = e?.statusCode;
+  const status = e?.statusCode ?? e?.status;
   const msg = String(e?.message ?? e);
-  console.error("\nERROR:", msg.slice(0, 300));
-  if (status === 401) {
-    console.error("=> The key is wrong or revoked. Check OPENAI_API_KEY.");
+  console.error("\nERROR:", msg.slice(0, 400));
+  const aad = /AADSTS\d+/.exec(msg)?.[0];
+  if (aad || /CredentialUnavailable|EnvironmentCredential/i.test(msg)) {
+    console.error(`=> Entra ID login failed${aad ? ` (${aad})` : ""}.`);
+    console.error("   Check AZURE_TENANT_ID, AZURE_CLIENT_ID, AZURE_CLIENT_SECRET (all three),"
+    );
+    console.error("   or run: az login --service-principal -u <clientId> -p <secret> --tenant <
+    tenantId>");
+    if (aad === "AADSTS7000215") console.error("   AADSTS7000215 = the client secret is wrong o
+    r expired.");
+    if (aad === "AADSTS700016") console.error("   AADSTS700016 = client id not found in that te
+    nant.");
+    if (aad === "AADSTS90002") console.error("   AADSTS90002 = tenant id not found.");
+  } else if (status === 401 || status === 403) {
+    if (PROVIDER !== "azure") {
+      console.error("=> The key is wrong or revoked. Check OPENAI_API_KEY.");
+    } else if (process.env.AZURE_OPENAI_API_KEY) {
+      console.error("=> Azure rejected the key. Check AZURE_OPENAI_API_KEY and AZURE_OPENAI_END
+    POINT.");
+    } else {
+      console.error("=> Entra issued a token but the resource refused it. The service principal
+     needs\n"
+        + "   the role 'Cognitive Services OpenAI User' on the Azure OpenAI resource (ask the a
+    dmin).");
+    }
   } else if (status === 429) {
-    console.error("=> Rate limit or no credit left (insufficient_quota). Check billing.");
+    console.error("=> Rate limit or quota exhausted. Wait, or ask for more TPM on the deploymen
+    t.");
   } else if (status === 404) {
-    console.error(`=> Model '${MODEL_ID}' not found. Try OPENAI_MODEL=gpt-4o-mini.`);
-  } else if (/ENOTFOUND|ECONNRESET|ETIMEDOUT|fetch failed/i.test(msg)) {
-    console.error("=> Network. Set HTTPS_PROXY if your company uses a proxy.");
+    console.error(PROVIDER === "azure"
+      ? `=> Not found. Is AZURE_OPENAI_DEPLOYMENT='${MODEL_ID}' the exact deployment name,\n`
+        + `   and is AZURE_OPENAI_ENDPOINT the resource URL (${AZURE_ENDPOINT})?`
+      : `=> Model '${MODEL_ID}' not found. Try OPENAI_MODEL=gpt-4o-mini.`);
+  } else if (/ENOTFOUND|ECONNRESET|ETIMEDOUT|fetch failed|getaddrinfo/i.test(msg)) {
+    console.error("=> Network. Set HTTPS_PROXY if your company uses a proxy; check the endpoint
+     host.");
   }
```

```
 }
--- a/labs/openai/00-check-key.ts
+++ b/labs/openai/00-check-key.ts
@@ -1,25 +1,57 @@
-// Step 0: prove the key works and see how many tokens a tiny call costs.
-//   OPENAI_API_KEY=sk-... bun run labs/openai/00-check-key.ts
+// Step 0: prove the credentials work and see how many tokens a tiny call costs.
+//   OpenAI : OPENAI_API_KEY=sk-... bun run labs/openai/00-check-key.ts
+//   Azure  : export AZURE_OPENAI_ENDPOINT, AZURE_OPENAI_DEPLOYMENT,
+//            AZURE_TENANT_ID, AZURE_CLIENT_ID, AZURE_CLIENT_SECRET
+//            bun run labs/openai/00-check-key.ts
 import { generateText } from "ai";
-import { createOpenAI } from "@ai-sdk/openai";
-import { MODEL_ID, explainError } from "./_model";
+import {
+  PROVIDER, MODEL_ID, AZURE_ENDPOINT, azureBaseURL, azureTokenProvider,
+  describeMode, realModel, explainError,
+} from "./_model";
-const key = process.env.OPENAI_API_KEY;
-if (!key) {
-  console.error("OPENAI_API_KEY is not set. In Git Bash:  export OPENAI_API_KEY=sk-...");
+function mask(v: string | undefined) {
+  return v ? `${v.slice(0, 4)}...${v.slice(-4)}` : "(not set)";
+}
+
+console.log(`mode: ${describeMode()}`);
+if (PROVIDER === "azure") {
+  console.log(`endpoint  : ${AZURE_ENDPOINT}  ->  ${azureBaseURL(AZURE_ENDPOINT!)}`);
+  console.log(`deployment: ${MODEL_ID || "(not set)"}`);
+  console.log(`tenant    : ${mask(process.env.AZURE_TENANT_ID)}`);
+  console.log(`client id : ${mask(process.env.AZURE_CLIENT_ID)}`);
+  console.log(`secret    : ${process.env.AZURE_CLIENT_SECRET ? "(set)" : "(not set)"}`);
+  if (!MODEL_ID) {
+    console.error("AZURE_OPENAI_DEPLOYMENT is not set. Ask the admin for the deployment name.")
   ;
+    process.exit(1);
+  }
+  if (!process.env.AZURE_OPENAI_API_KEY) {
+    // Step A: get a token from Entra ID. This proves tenant/client/secret.
+    try {
+      const t0 = Date.now();
+      const token = await azureTokenProvider()();
+      console.log(`entra token: OK, ${token.length} chars (${Date.now() - t0} ms)`);
+    } catch (e) {
+      explainError(e);
+      process.exit(1);
+    }
+  }
+} else if (PROVIDER === "openai") {
+  console.log(`key: ${mask(process.env.OPENAI_API_KEY)}  model: ${MODEL_ID}`);
+} else {
+  console.log("MOCK=1 makes no sense here: this script exists to test real credentials.");
   process.exit(1);
 }
-console.log(`key: ${key.slice(0, 7)}...${key.slice(-4)}  model: ${MODEL_ID}`);
+// Step B: one tiny completion. This proves the role on the resource and the deployment name.
 try {
   const t0 = Date.now();
   const r = await generateText({
-    model: createOpenAI({ apiKey: key })(MODEL_ID),
+    model: realModel(),
     prompt: "Reply with exactly the two letters: OK",
   });
   console.log(`reply: ${JSON.stringify(r.text)}  (${Date.now() - t0} ms)`);
   console.log(`tokens: input=${r.usage.inputTokens} output=${r.usage.outputTokens}`);
-  console.log("Key works. You can run the other labs without MOCK=1.");
+  console.log("Credentials work. You can run the other labs without MOCK=1.");
 } catch (e) {
   explainError(e);
   process.exit(1);
--- a/labs/openai/03-server.ts
+++ b/labs/openai/03-server.ts
@@ -1,14 +1,13 @@
 // Lab 3: a localhost chat server. The gate is stateless; state comes from message history.
 //   rehearse:  MOCK=1 bun run labs/openai/03-server.ts
-//   real:      OPENAI_API_KEY=sk-... bun run labs/openai/03-server.ts
+//   real:      OPENAI_API_KEY=sk-... (or the AZURE_* vars) bun run labs/openai/03-server.ts
 // Then, in a second Git Bash window:  bun run labs/openai/03-client.ts
 import { generateText, tool, stepCountIs } from "ai";
 import type { ModelMessage } from "ai";
 import { z } from "zod";
 import { compile } from "@petriflow/rules";
 import { createPetriflowGate } from "@petriflow/vercel-ai";
-import { createOpenAI } from "@ai-sdk/openai";
```

```
-import { MOCK, MODEL_ID, scriptedModel, banner } from "./_model";
+import { MOCK, scriptedModel, realModel, describeMode, banner } from "./_model";
 banner("Lab 3: localhost server with replay");
 const PORT = Number(process.env.PORT ?? 3456);
@@ -43,7 +42,7 @@
 // 3. Pick the model. The mock reacts to the last user message.
 function modelFor(prompt: string) {
-  if (!MOCK) return createOpenAI({ apiKey: process.env.OPENAI_API_KEY ?? "" })(MODEL_ID);
+  if (!MOCK) return realModel();
   const p = prompt.toLowerCase();
   if (p.includes("test")) {
     return scriptedModel([{ tool: "test" }, { text: "Tests passed." }]);
@@ -57,10 +56,7 @@
   return scriptedModel([{ text: "I can run tests or deploy." }]);
 }
-if (!MOCK && !process.env.OPENAI_API_KEY) {
-  console.error("OPENAI_API_KEY is not set. Export it, or run with MOCK=1.");
-  process.exit(1);
-}
+if (!MOCK) realModel(); // fail fast if the provider config is missing
 const SYSTEM_PREFIX = "You are a release assistant. Use the tools. "
   + "If a tool is blocked, explain the reason.\n\n";
@@ -71,7 +67,7 @@
   async fetch(req) {
     const url = new URL(req.url);
     if (req.method === "GET" && url.pathname === "/") {
-      const mode = MOCK ? "mock" : `openai:${MODEL_ID}`;
+      const mode = describeMode();
       return Response.json({ ok: true, rules: verification, mode });
     }
     if (req.method === "POST" && url.pathname === "/chat") {
--- a/labs/openai/04-decision-provider.ts
+++ b/labs/openai/04-decision-provider.ts
@@ -1,14 +1,13 @@
 // Lab 4: the engine side. OpenAI chooses which transition fires at a choice point.
 //   rehearse:  MOCK=1 bun run labs/openai/04-decision-provider.ts
-//   real:      OPENAI_API_KEY=sk-... bun run labs/openai/04-decision-provider.ts
+//   real:      OPENAI_API_KEY=sk-... (or the AZURE_* vars) bun run labs/openai/04-decision-pro
   vider.ts
 import { generateText, tool } from "ai";
 import { z } from "zod";
 import { Database } from "bun:sqlite";
 import { Scheduler, createExecutor, sqliteAdapter } from "@petriflow/engine";
 import type { DecisionProvider } from "@petriflow/engine/decision";
-import { createOpenAI } from "@ai-sdk/openai";
 import { definition } from "../../workflows/simple-agent/definition";
-import { MOCK, MODEL_ID, banner, explainError } from "./_model";
+import { MOCK, realModel, banner, explainError } from "./_model";
 banner("Lab 4: OpenAI as the engine's decision provider");
@@ -37,7 +36,7 @@
       .map((t) => `${t.name} [${t.inputs}] -> [${t.outputs}]`)
       .join("; ");
     const result = await generateText({
-      model: createOpenAI({ apiKey: process.env.OPENAI_API_KEY ?? "" })(MODEL_ID),
+      model: realModel(),
       system: `You control the workflow "${req.workflowName}". `
         + "Prefer to iterate once to gather more tool results, then generate. "
         + "Pick exactly one transition.",
@@ -56,10 +55,7 @@
   },
 };
-if (!MOCK && !process.env.OPENAI_API_KEY) {
-  console.error("OPENAI_API_KEY is not set. Export it, or run with MOCK=1.");
-  process.exit(1);
-}
+if (!MOCK) realModel(); // fail fast if the provider config is missing
 const db = new Database(":memory:");
 const executor = createExecutor(definition as any, {
--- a/labs/openai/README.md
+++ b/labs/openai/README.md
@@ -1,25 +1,43 @@
-# OpenAI labs for PetriFlow
+# LLM labs for PetriFlow (OpenAI or Azure OpenAI)
-Four runnable agents that drive the PetriFlow gate with an OpenAI model, plus a key check.
+Four runnable agents that drive the PetriFlow gate with a real model, plus a credentials check.
 Every lab also runs with `MOCK=1`: a scripted fake model, no network, no cost.
-Setup (once, at the repo root): `bun add @ai-sdk/openai@^3`
-Revert later with `git checkout -- package.json bun.lock`.
+Setup (once, at the repo root):
+- `bun add @ai-sdk/openai@^3 @ai-sdk/azure@^3 @azure/identity`
+- Revert later with `git checkout -- package.json bun.lock`.
+
+Provider is chosen from the environment, in this order:
+1. `MOCK=1`                    scripted model (no network)
```

```
+2. `AZURE_OPENAI_ENDPOINT` set Azure OpenAI
+3. otherwise                   OpenAI with `OPENAI_API_KEY`
+
+Azure OpenAI with a service principal (what a company usually hands out):
+- `AZURE_OPENAI_ENDPOINT`    e.g. `https://<resource>.openai.azure.com`
+- `AZURE_OPENAI_DEPLOYMENT`  the deployment name the admin created (not the model family)
+- `AZURE_TENANT_ID`, `AZURE_CLIENT_ID`, `AZURE_CLIENT_SECRET`  the service principal
+- optional `AZURE_OPENAI_API_KEY`  if you were given a key instead of a principal
+- optional `AZURE_OPENAI_SCOPE`    default `https://cognitiveservices.azure.com/.default`
+- The principal needs the role "Cognitive Services OpenAI User" on the resource.
+- Alternative to the three AZURE_* auth vars: `az login --service-principal ...`
+  (DefaultAzureCredential picks up the Azure CLI login).
-Environment variables:
-- `OPENAI_API_KEY`  required for real mode
+OpenAI:
+- `OPENAI_API_KEY`  required
 - `OPENAI_MODEL`    default `gpt-4o-mini`
-- `MOCK=1`          use the scripted model instead of OpenAI
+
+Other variables:
 - `WORKDIR`         lab 1 scratch folder, default `C:/tmp/petriflow-lab`
 - `PORT` / `BASE`   lab 3 server port (default 3456) and client base URL
 Files:
-- `00-check-key.ts`         key works, token usage (real mode only)
+- `_model.ts`               provider switch: getModel / realModel / scriptedModel
+- `00-check-key.ts`         credentials work, token usage (real mode only)
 - `01-file-agent.ts`        require backup before delete, block rm
 - `02-deploy-agent.ts`      lint -> test -> deploy, human approval (y/n), limit 2
 - `03-server.ts`            localhost server, per-request sessions, replay from history
 - `03-client.ts`            sends three requests to the server
-- `04-decision-provider.ts` OpenAI picks transitions inside the workflow engine
+- `04-decision-provider.ts` the model picks transitions inside the workflow engine
 Rehearse for free:
 - `MOCK=1 bun run labs/openai/01-file-agent.ts`
@@ -27,4 +45,4 @@
 - `MOCK=1 bun run labs/openai/03-server.ts` then `bun run labs/openai/03-client.ts`
 - `MOCK=1 bun run labs/openai/04-decision-provider.ts`
-Real: export `OPENAI_API_KEY`, drop `MOCK=1`, start with `00-check-key.ts`.
+Real: export the variables for your provider, drop `MOCK=1`, start with `00-check-key.ts`.
diff --git a/package.json b/package.json
index 0942c7e..4579b9d 100644
--- a/package.json
+++ b/package.json
@@ -18,6 +18,9 @@
     "@types/bun": "^1"
   },
   "dependencies": {
+    "@ai-sdk/azure": "^3",
+    "@ai-sdk/openai": "^3",
+    "@azure/identity": "^4.13.3",
     "ai": "^6.0.86"
   }
 }
```

Appendix B. The changed files, paste-ready

Each block recreates one file. Paste into Git Bash at the repo root, from the `cat` line to the `EOF` line. Files not listed here (01, 02, 03-client) are unchanged from the OpenAI guide.

labs/openai/_model.ts

The plug: mock / Azure / OpenAI selection, token provider, error hints.

```
cat > labs/openai/_model.ts <<'EOF'
// Shared helper for the LLM labs. Three ways to get a model, picked by env vars:
//
//   mock mode  : MOCK=1. A scripted fake model that returns the tool calls you give it.
//                No network, no key, no cost. Use it to rehearse a lab before paying.
//   azure mode : AZURE_OPENAI_ENDPOINT is set. Auth is a service principal
//                (AZURE_TENANT_ID / AZURE_CLIENT_ID / AZURE_CLIENT_SECRET) via Entra ID,
//                or AZURE_OPENAI_API_KEY if you were given a key instead.
//   openai mode: otherwise. Needs OPENAI_API_KEY. Model from OPENAI_MODEL.
import { createOpenAI } from "@ai-sdk/openai";
import { createAzure } from "@ai-sdk/azure";
import { DefaultAzureCredential, getBearerTokenProvider } from "@azure/identity";
import { MockLanguageModelV3 } from "ai/test";
import type { LanguageModel } from "ai";
export const MOCK = process.env.MOCK === "1";
export const AZURE_ENDPOINT = process.env.AZURE_OPENAI_ENDPOINT;
export const PROVIDER: "mock" | "azure" | "openai" =
  MOCK ? "mock" : AZURE_ENDPOINT ? "azure" : "openai";
// Azure: the "model" is the DEPLOYMENT NAME your admin created, not the model family.
export const MODEL_ID = PROVIDER === "azure"
  ? (process.env.AZURE_OPENAI_DEPLOYMENT ?? "")
  : (process.env.OPENAI_MODEL ?? "gpt-4o-mini");
// Token audience for Azure OpenAI. Rarely needs changing.
const AZURE_SCOPE = process.env.AZURE_OPENAI_SCOPE ?? "https://cognitiveservices.azure.com/.default";
export type PlanStep =
  | { tool: string; input?: Record<string, unknown> }
  | { text: string };
function usage(input: number, output: number) {
  return {
    inputTokens: { total: input, noCache: input, cacheRead: 0, cacheWrite: 0 },
    outputTokens: { total: output, text: output, reasoning: 0 },
    raw: undefined,
  };
}
/** A fake model that replays a fixed plan: one step per model call. */
export function scriptedModel(plan: PlanStep[]): LanguageModel {
  let i = 0;
  return new MockLanguageModelV3({
    provider: "mock",
    modelId: "scripted",
    doGenerate: async () => {
      const step = plan[Math.min(i, plan.length - 1)]!;
      i++;
      if ("text" in step) {
        return {
          content: [{ type: "text", text: step.text }],
          finishReason: { unified: "stop", raw: undefined },
          usage: usage(0, 0),
          warnings: [],
        };
      }
      return {
        content: [{
          type: "tool-call",
          toolCallId: `mock-${i}`,
          toolName: step.tool,
          input: JSON.stringify(step.input ?? {}),
        }],
        finishReason: { unified: "tool-calls", raw: undefined },
        usage: usage(0, 0),
        warnings: [],
```

```
      };
    },
  }) as unknown as LanguageModel;
}
function die(msg: string): never {
  console.error(msg);
  process.exit(1);
}
/** Entra ID token source. DefaultAzureCredential reads AZURE_TENANT_ID, AZURE_CLIENT_ID
 *  and AZURE_CLIENT_SECRET (or AZURE_CLIENT_CERTIFICATE_PATH), and falls back to
 *  `az login` if those are absent. Tokens are cached and refreshed automatically. */
export function azureTokenProvider() {
  return getBearerTokenProvider(new DefaultAzureCredential(), AZURE_SCOPE);
}
/** Normalise what the admin gave you into a base URL the AI SDK accepts.
 *  "https://foo.openai.azure.com" -> "https://foo.openai.azure.com/openai/v1"
 *  Anything with a path (gateway / APIM / already /openai/v1) is kept as-is. */
export function azureBaseURL(endpoint: string): string {
  const trimmed = endpoint.replace(/\/+$/, "");
  const path = new URL(trimmed).pathname.replace(/\/+$/, "");
  return path === "" ? `${trimmed}/openai/v1` : trimmed;
}
/** The real model for the current provider. Exits with a clear message if config is missing. */
export function realModel(): LanguageModel {
  if (PROVIDER === "azure") {
    if (!MODEL_ID) die("AZURE_OPENAI_DEPLOYMENT is not set (the deployment name, e.g. gpt-4o-mini).");
    const apiKey = process.env.AZURE_OPENAI_API_KEY;
    const azure = createAzure(apiKey
      ? { baseURL: azureBaseURL(AZURE_ENDPOINT!), apiKey }
      : { baseURL: azureBaseURL(AZURE_ENDPOINT!), tokenProvider: azureTokenProvider() });
    return azure(MODEL_ID);
  }
  const key = process.env.OPENAI_API_KEY;
  if (!key) die("OPENAI_API_KEY is not set. Export it, set AZURE_OPENAI_ENDPOINT, or run with MOCK=1.");
  return createOpenAI({ apiKey: key })(MODEL_ID);
}
/** Real model, or the scripted mock when MOCK=1. */
export function getModel(plan: PlanStep[]): LanguageModel {
  if (MOCK) return scriptedModel(plan);
  return realModel();
}
export function describeMode(): string {
  if (PROVIDER === "mock") return "MOCK (scripted model, no network)";
  if (PROVIDER === "azure") {
    const auth = process.env.AZURE_OPENAI_API_KEY ? "api key" : "service principal (Entra ID)";
    return `REAL (Azure OpenAI, deployment ${MODEL_ID || "?"}, ${auth})`;
  }
  return `REAL (OpenAI ${MODEL_ID})`;
}
export function banner(title: string) {
  console.log(`=== ${title} ===`);
  console.log(`mode: ${describeMode()}`);
  console.log("");
}
/** Print every step of a generateText result: tool calls, tool results, text. */
export function printSteps(result: { steps: any[]; text: string; usage?: any }) {
  console.log("\n--- Steps ---");
  result.steps.forEach((s, n) => {
    const parts: any[] = s.content ?? [];
    for (const p of parts) {
      if (p.type === "tool-call") {
        console.log(`${n + 1}. model called ${p.toolName}(${JSON.stringify(p.input)})`);
      } else if (p.type === "tool-result") {
        console.log(`   -> result: ${JSON.stringify(p.output).slice(0, 120)}`);
      } else if (p.type === "tool-error") {
        const msg = String(p.error?.message ?? p.error).slice(0, 140);
        console.log(`   -> ERROR returned to model: ${msg}`);
      } else if (p.type === "text" && p.text) {
        console.log(`   model said: ${p.text.slice(0, 200)}`);
      }
    }
  });
  console.log("\n--- Final answer ---");
```

```
  console.log(result.text || "(no text)");
  if (result.usage) {
    const u = result.usage;
    console.log(`\ntokens: input=${u.inputTokens ?? "?"} output=${u.outputTokens ?? "?"}`);
  }
}
/** Friendly explanation of common OpenAI / Azure errors. */
export function explainError(e: any) {
  const status = e?.statusCode ?? e?.status;
  const msg = String(e?.message ?? e);
  console.error("\nERROR:", msg.slice(0, 400));
  const aad = /AADSTS\d+/.exec(msg)?.[0];
  if (aad || /CredentialUnavailable|EnvironmentCredential/i.test(msg)) {
    console.error(`=> Entra ID login failed${aad ? ` (${aad})` : ""}.`);
    console.error("   Check AZURE_TENANT_ID, AZURE_CLIENT_ID, AZURE_CLIENT_SECRET (all three),");
    console.error("   or run: az login --service-principal -u <clientId> -p <secret> --tenant <tenantId
    if (aad === "AADSTS7000215") console.error("   AADSTS7000215 = the client secret is wrong or expire
    if (aad === "AADSTS700016") console.error("   AADSTS700016 = client id not found in that tenant.");
    if (aad === "AADSTS90002") console.error("   AADSTS90002 = tenant id not found.");
  } else if (status === 401 || status === 403) {
    if (PROVIDER !== "azure") {
      console.error("=> The key is wrong or revoked. Check OPENAI_API_KEY.");
    } else if (process.env.AZURE_OPENAI_API_KEY) {
      console.error("=> Azure rejected the key. Check AZURE_OPENAI_API_KEY and AZURE_OPENAI_ENDPOINT.")
    } else {
      console.error("=> Entra issued a token but the resource refused it. The service principal needs\r
        + "   the role 'Cognitive Services OpenAI User' on the Azure OpenAI resource (ask the admin).")
    }
  } else if (status === 429) {
    console.error("=> Rate limit or quota exhausted. Wait, or ask for more TPM on the deployment.");
  } else if (status === 404) {
    console.error(PROVIDER === "azure"
      ? `=> Not found. Is AZURE_OPENAI_DEPLOYMENT='${MODEL_ID}' the exact deployment name,\n`
        + `   and is AZURE_OPENAI_ENDPOINT the resource URL (${AZURE_ENDPOINT})?`
      : `=> Model '${MODEL_ID}' not found. Try OPENAI_MODEL=gpt-4o-mini.`);
  } else if (/ENOTFOUND|ECONNRESET|ETIMEDOUT|fetch failed|getaddrinfo/i.test(msg)) {
    console.error("=> Network. Set HTTPS_PROXY if your company uses a proxy; check the endpoint host.")
  }
}
EOF
```

Paste everything including the last EOF line.

labs/openai/00-check-key.ts

Lab 0: Entra token, then one tiny completion.

```
cat > labs/openai/00-check-key.ts <<'EOF'
// Step 0: prove the credentials work and see how many tokens a tiny call costs.
//   OpenAI : OPENAI_API_KEY=sk-... bun run labs/openai/00-check-key.ts
//   Azure  : export AZURE_OPENAI_ENDPOINT, AZURE_OPENAI_DEPLOYMENT,
//            AZURE_TENANT_ID, AZURE_CLIENT_ID, AZURE_CLIENT_SECRET
//            bun run labs/openai/00-check-key.ts
import { generateText } from "ai";
import {
  PROVIDER, MODEL_ID, AZURE_ENDPOINT, azureBaseURL, azureTokenProvider,
  describeMode, realModel, explainError,
} from "./_model";
function mask(v: string | undefined) {
  return v ? `${v.slice(0, 4)}...${v.slice(-4)}` : "(not set)";
}
console.log(`mode: ${describeMode()}`);
if (PROVIDER === "azure") {
  console.log(`endpoint  : ${AZURE_ENDPOINT}  ->  ${azureBaseURL(AZURE_ENDPOINT!)}`);
  console.log(`deployment: ${MODEL_ID || "(not set)"}`);
  console.log(`tenant    : ${mask(process.env.AZURE_TENANT_ID)}`);
  console.log(`client id : ${mask(process.env.AZURE_CLIENT_ID)}`);
  console.log(`secret    : ${process.env.AZURE_CLIENT_SECRET ? "(set)" : "(not set)"}`);
  if (!MODEL_ID) {
    console.error("AZURE_OPENAI_DEPLOYMENT is not set. Ask the admin for the deployment name.");
    process.exit(1);
  }
```

```
  if (!process.env.AZURE_OPENAI_API_KEY) {
    // Step A: get a token from Entra ID. This proves tenant/client/secret.
    try {
      const t0 = Date.now();
      const token = await azureTokenProvider()();
      console.log(`entra token: OK, ${token.length} chars (${Date.now() - t0} ms)`);
    } catch (e) {
      explainError(e);
      process.exit(1);
    }
  }
} else if (PROVIDER === "openai") {
  console.log(`key: ${mask(process.env.OPENAI_API_KEY)}  model: ${MODEL_ID}`);
} else {
  console.log("MOCK=1 makes no sense here: this script exists to test real credentials.");
  process.exit(1);
}
// Step B: one tiny completion. This proves the role on the resource and the deployment name.
try {
  const t0 = Date.now();
  const r = await generateText({
    model: realModel(),
    prompt: "Reply with exactly the two letters: OK",
  });
  console.log(`reply: ${JSON.stringify(r.text)}  (${Date.now() - t0} ms)`);
  console.log(`tokens: input=${r.usage.inputTokens} output=${r.usage.outputTokens}`);
  console.log("Credentials work. You can run the other labs without MOCK=1.");
} catch (e) {
  explainError(e);
  process.exit(1);
}
EOF
```

Paste everything including the last EOF line.

labs/openai/03-server.ts

Lab 3 server, now via realModel().

```
cat > labs/openai/03-server.ts <<'EOF'
// Lab 3: a localhost chat server. The gate is stateless; state comes from message history.
//   rehearse:  MOCK=1 bun run labs/openai/03-server.ts
//   real:      OPENAI_API_KEY=sk-... (or the AZURE_* vars) bun run labs/openai/03-server.ts
// Then, in a second Git Bash window:  bun run labs/openai/03-client.ts
import { generateText, tool, stepCountIs } from "ai";
import type { ModelMessage } from "ai";
import { z } from "zod";
import { compile } from "@petriflow/rules";
import { createPetriflowGate } from "@petriflow/vercel-ai";
import { MOCK, scriptedModel, realModel, describeMode, banner } from "./_model";
banner("Lab 3: localhost server with replay");
const PORT = Number(process.env.PORT ?? 3456);
// 1. Compile once at startup. The gate object holds no per-user state.
const { nets, verification } = compile(`
  require test before deploy
  limit deploy to 2 per session
`);
const gate = createPetriflowGate(nets, {
  isToolResultError: (_n, r) => (r as any)?.passed === false,
  onDecision: (event, decision) => {
    console.log(decision?.block
      ? `  [gate] BLOCKED ${event.toolName}: ${decision.reason}`
      : `  [gate] allow ${event.toolName}`);
  },
});
// 2. Tools shared by every request (they are pure, so sharing is safe).
const tools = {
  test: tool({
    description: "Run the test suite",
    inputSchema: z.object({}),
    execute: async () => ({ passed: true }),
  }),
```

```
  deploy: tool({
    description: "Deploy a version to production",
    inputSchema: z.object({ version: z.string() }),
    execute: async ({ version }) => ({ deployed: version }),
  }),
};
// 3. Pick the model. The mock reacts to the last user message.
function modelFor(prompt: string) {
  if (!MOCK) return realModel();
  const p = prompt.toLowerCase();
  if (p.includes("test")) {
    return scriptedModel([{ tool: "test" }, { text: "Tests passed." }]);
  }
  if (p.includes("deploy")) {
    return scriptedModel([
      { tool: "deploy", input: { version: "v1" } },
      { text: "Deploy attempted." },
    ]);
  }
  return scriptedModel([{ text: "I can run tests or deploy." }]);
}
if (!MOCK) realModel(); // fail fast if the provider config is missing
const SYSTEM_PREFIX = "You are a release assistant. Use the tools. "
  + "If a tool is blocked, explain the reason.\n\n";
// 4. The server. POST /chat {prompt, messages?} -> {text, blocked, status, messages}
const server = Bun.serve({
  port: PORT,
  async fetch(req) {
    const url = new URL(req.url);
    if (req.method === "GET" && url.pathname === "/") {
      const mode = describeMode();
      return Response.json({ ok: true, rules: verification, mode });
    }
    if (req.method === "POST" && url.pathname === "/chat") {
      const body = (await req.json()) as { prompt: string; messages?: ModelMessage[] };
      const history: ModelMessage[] = body.messages ?? [];
      console.log(`\nPOST /chat  prompt="${body.prompt}"  history=${history.length}`);
      // Fresh session per request. Given history, replay advances the nets first.
      const session = gate.wrapTools(tools, { messages: history });
      console.log("  state after replay:", session.formatStatus().replace(/\n/g, " | "));
      const blocked: string[] = [];
      const messages: ModelMessage[] = [...history, { role: "user", content: body.prompt }];
      try {
        const result = await generateText({
          model: modelFor(body.prompt),
          system: SYSTEM_PREFIX + session.systemPrompt(),
          tools: session.tools,
          messages,
          stopWhen: stepCountIs(6),
        });
        for (const s of result.steps) {
          for (const p of (s.content ?? []) as any[]) {
            if (p.type === "tool-error") blocked.push(String(p.error?.message ?? p.error));
          }
        }
        return Response.json({
          text: result.text,
          blocked,
          status: session.formatStatus(),
          messages: [...messages, ...result.response.messages],
        });
      } catch (e: any) {
        const error = String(e?.message ?? e);
        return Response.json({ error, status: e?.statusCode ?? 500 }, { status: 500 });
      }
    }
    return new Response("not found", { status: 404 });
  },
});
console.log(`listening on http://localhost:${server.port}`);
console.log(`try:  curl http://localhost:${server.port}/`);
console.log("then: bun run labs/openai/03-client.ts   (other window). Ctrl+C stops.");
EOF
```

Paste everything including the last EOF line.

labs/openai/04-decision-provider.ts

Lab 4, now via realModel().

```
cat > labs/openai/04-decision-provider.ts <<'EOF'
// Lab 4: the engine side. OpenAI chooses which transition fires at a choice point.
//   rehearse:  MOCK=1 bun run labs/openai/04-decision-provider.ts
//   real:      OPENAI_API_KEY=sk-... (or the AZURE_* vars) bun run labs/openai/04-decision-provider.ts
import { generateText, tool } from "ai";
import { z } from "zod";
import { Database } from "bun:sqlite";
import { Scheduler, createExecutor, sqliteAdapter } from "@petriflow/engine";
import type { DecisionProvider } from "@petriflow/engine/decision";
import { definition } from "../../workflows/simple-agent/definition";
import { MOCK, realModel, banner, explainError } from "./_model";
banner("Lab 4: OpenAI as the engine's decision provider");
type Provider = DecisionProvider<string, Record<string, unknown>>;
// The mock chooses 'iterate' while budget remains, then 'generate'.
const mockProvider: Provider = {
  async choose(req) {
    const names = req.enabled.map((t) => t.name);
    const budget = (req.marking as any).iterationBudget ?? 0;
    const pick = names.includes("iterate") && budget > 0 ? "iterate" : names[0]!;
    const reasoning = `mock: budget=${budget}, options=${names.join("/")}`;
    return { transition: pick, reasoning };
  },
};
// The real provider asks OpenAI to pick one enabled transition via a forced tool call.
const openaiProvider: Provider = {
  async choose(req) {
    const names = req.enabled.map((t) => t.name) as [string, ...string[]];
    const marking = Object.entries(req.marking)
      .filter(([, v]) => (v as number) > 0)
      .map(([k, v]) => `${k}=${v}`)
      .join(", ");
    const enabled = req.enabled
      .map((t) => `${t.name} [${t.inputs}] -> [${t.outputs}]`)
      .join("; ");
    const result = await generateText({
      model: realModel(),
      system: `You control the workflow "${req.workflowName}". `
        + "Prefer to iterate once to gather more tool results, then generate. "
        + "Pick exactly one transition.",
      prompt: `Marking: ${marking}\nContext: ${JSON.stringify(req.context)}\n`
        + `Enabled: ${enabled}`,
      tools: {
        pickTransition: tool({
          description: "Choose the transition to fire next",
          inputSchema: z.object({ transition: z.enum(names), reasoning: z.string() }),
        }),
      },
      toolChoice: { type: "tool", toolName: "pickTransition" },
    });
    const input = result.toolCalls[0]!.input as any;
    return { transition: input.transition, reasoning: input.reasoning };
  },
};
if (!MOCK) realModel(); // fail fast if the provider config is missing
const db = new Database(":memory:");
const executor = createExecutor(definition as any, {
  decisionProvider: MOCK ? mockProvider : openaiProvider,
});
const scheduler = new Scheduler(executor, { adapter: sqliteAdapter(db, definition.name) }, {
  onDecision: (_id, name, reasoning, candidates) => {
    console.log(`  [llm] chose ${name} from [${candidates.join(", ")}]: ${reasoning}`);
  },
  onFire: (_id, name) => console.log(`fired: ${name}`),
  onComplete: (id) => console.log(`${id} completed`),
  onError: (_id, err) => explainError(err),
```

```
});
await scheduler.createInstance("agent-llm");
for (let i = 0; i < 40; i++) {
  const n = await scheduler.tick();
  if (n === 0) break;
}
const state = await scheduler.inspect("agent-llm");
console.log("\nfinal:", JSON.stringify({
  marking: state.marking,
  iteration: (state.context as any).iteration,
  status: state.status,
}));
EOF
```

Paste everything including the last EOF line.

labs/openai/README.md

Folder index with the Azure variables.

```
cat > labs/openai/README.md <<'EOF'
# LLM labs for PetriFlow (OpenAI or Azure OpenAI)
Four runnable agents that drive the PetriFlow gate with a real model, plus a credentials check.
Every lab also runs with `MOCK=1`: a scripted fake model, no network, no cost.
Setup (once, at the repo root):
- `bun add @ai-sdk/openai@^3 @ai-sdk/azure@^3 @azure/identity`
- Revert later with `git checkout -- package.json bun.lock`.
Provider is chosen from the environment, in this order:
1. `MOCK=1`                    scripted model (no network)
2. `AZURE_OPENAI_ENDPOINT` set Azure OpenAI
3. otherwise                   OpenAI with `OPENAI_API_KEY`
Azure OpenAI with a service principal (what a company usually hands out):
- `AZURE_OPENAI_ENDPOINT`    e.g. `https://<resource>.openai.azure.com`
- `AZURE_OPENAI_DEPLOYMENT`  the deployment name the admin created (not the model family)
- `AZURE_TENANT_ID`, `AZURE_CLIENT_ID`, `AZURE_CLIENT_SECRET`  the service principal
- optional `AZURE_OPENAI_API_KEY`  if you were given a key instead of a principal
- optional `AZURE_OPENAI_SCOPE`    default `https://cognitiveservices.azure.com/.default`
- The principal needs the role "Cognitive Services OpenAI User" on the resource.
- Alternative to the three AZURE_* auth vars: `az login --service-principal ...`
  (DefaultAzureCredential picks up the Azure CLI login).
OpenAI:
- `OPENAI_API_KEY`  required
- `OPENAI_MODEL`    default `gpt-4o-mini`
Other variables:
- `WORKDIR`         lab 1 scratch folder, default `C:/tmp/petriflow-lab`
- `PORT` / `BASE`   lab 3 server port (default 3456) and client base URL
Files:
- `_model.ts`               provider switch: getModel / realModel / scriptedModel
- `00-check-key.ts`         credentials work, token usage (real mode only)
- `01-file-agent.ts`        require backup before delete, block rm
- `02-deploy-agent.ts`      lint -> test -> deploy, human approval (y/n), limit 2
- `03-server.ts`            localhost server, per-request sessions, replay from history
- `03-client.ts`            sends three requests to the server
- `04-decision-provider.ts` the model picks transitions inside the workflow engine
Rehearse for free:
- `MOCK=1 bun run labs/openai/01-file-agent.ts`
- `MOCK=1 AUTO_APPROVE=1 bun run labs/openai/02-deploy-agent.ts`
- `MOCK=1 bun run labs/openai/03-server.ts` then `bun run labs/openai/03-client.ts`
- `MOCK=1 bun run labs/openai/04-decision-provider.ts`
Real: export the variables for your provider, drop `MOCK=1`, start with `00-check-key.ts`.
EOF
```

Paste everything including the last EOF line.

Appendix C. The prompt file

Creates `labs/openai/COPILOT_PROMPT_AZURE.md` so you can open it in VS Code and copy the prompt without PDF artefacts.

```
cat > labs/openai/COPILOT_PROMPT_AZURE.md <<'EOF'
# Copilot prompt: Azure OpenAI with a service principal for the PetriFlow labs
Paste everything below the line into GitHub Copilot Chat (agent mode) with the repo open.
---
You are working in the petri-flow repository: a Bun + TypeScript monorepo. Goal: let the
scripts in `labs/openai/` talk to **Azure OpenAI authenticated with a Microsoft Entra ID
service principal**, with the smallest possible change. Do NOT modify anything under
`packages/`, `workflows/`, `apps/`, `site/`, `comparisons/` or `examples/`. The PetriFlow
gate and engine are provider-agnostic; only the place where the model object is built changes.
## Context you can rely on
- The labs use the Vercel AI SDK (`ai@6`). A model is a `LanguageModel` object; the gate never
  sees it. `labs/openai/_model.ts` builds the model in `getModel(plan)`. Three scripts still
  call `createOpenAI` directly: `00-check-key.ts`, `03-server.ts`, `04-decision-provider.ts`.
- `@ai-sdk/azure@3` exports `createAzure(options)`. Relevant options: `baseURL`, `apiKey`,
  `tokenProvider: () => Promise<string>` (called on every request; the SDK adds
  `Authorization: Bearer <token>`). `apiKey` and `tokenProvider` must not both be set.
  A base URL ending in `/openai/v1` is used as-is. `createAzure(...)("<deployment>")` returns
  the model; the id is the Azure **deployment name**, not the model family.
- `@azure/identity` exports `DefaultAzureCredential` (reads `AZURE_TENANT_ID`,
  `AZURE_CLIENT_ID`, `AZURE_CLIENT_SECRET` or `AZURE_CLIENT_CERTIFICATE_PATH`, and falls back
  to `az login`) and `getBearerTokenProvider(credential, scope)` which returns a cached,
  auto-refreshing `() => Promise<string>`. Scope: `https://cognitiveservices.azure.com/.default`.
## Steps
1. At the repo root run: `bun add @ai-sdk/azure@^3 @azure/identity`
2. Edit `labs/openai/_model.ts`:
   a. Add imports: `createAzure` from `@ai-sdk/azure`; `DefaultAzureCredential` and
      `getBearerTokenProvider` from `@azure/identity`.
   b. `export const AZURE_ENDPOINT = process.env.AZURE_OPENAI_ENDPOINT;`
      `export const PROVIDER: "mock" | "azure" | "openai" =
        MOCK ? "mock" : AZURE_ENDPOINT ? "azure" : "openai";`
   c. `MODEL_ID`: in azure mode use `process.env.AZURE_OPENAI_DEPLOYMENT ?? ""`, otherwise keep
      `process.env.OPENAI_MODEL ?? "gpt-4o-mini"`.
   d. `export function azureTokenProvider()` returning
      `getBearerTokenProvider(new DefaultAzureCredential(), process.env.AZURE_OPENAI_SCOPE ??
      "https://cognitiveservices.azure.com/.default")`.
   e. `export function azureBaseURL(endpoint: string): string`: strip trailing slashes; if the
      URL has an empty path, append `/openai/v1`; otherwise return it unchanged (gateways and
      full `/openai/v1` URLs pass through).
   f. `export function realModel(): LanguageModel`:
      - azure: if `MODEL_ID` is empty, print "AZURE_OPENAI_DEPLOYMENT is not set" and exit(1).
        If `AZURE_OPENAI_API_KEY` is set use `createAzure({ baseURL, apiKey })`, else
        `createAzure({ baseURL, tokenProvider: azureTokenProvider() })`. Return `azure(MODEL_ID)`.
      - openai: unchanged behaviour (exit with a message if `OPENAI_API_KEY` is missing;
        mention `AZURE_OPENAI_ENDPOINT` and `MOCK=1` as alternatives).
   g. `getModel(plan)` becomes `MOCK ? scriptedModel(plan) : realModel()`.
   h. `export function describeMode(): string` returning one line, e.g.
      "MOCK (scripted model, no network)",
      "REAL (Azure OpenAI, deployment <name>, service principal (Entra ID))",
      "REAL (OpenAI gpt-4o-mini)". `banner()` prints `mode: ${describeMode()}`.
   i. `explainError`: read status from `e.statusCode ?? e.status`. Add branches:
      - message contains `AADSTS\d+` or `CredentialUnavailable`/`EnvironmentCredential`:
        say Entra login failed, name the AADSTS code, tell the user to check the three
        AZURE_* variables or run `az login --service-principal`. Special-case AADSTS7000215
        (bad secret), AADSTS700016 (client id not in tenant), AADSTS90002 (tenant not found).
      - 401/403 in azure mode: with `AZURE_OPENAI_API_KEY` say the key or endpoint is wrong;
        otherwise say the principal needs the role "Cognitive Services OpenAI User" on the
        resource.
      - 404 in azure mode: say to check `AZURE_OPENAI_DEPLOYMENT` (exact deployment name) and
        `AZURE_OPENAI_ENDPOINT`.
      - keep the existing OpenAI branches and the network branch (add `getaddrinfo`).
3. Rewrite `labs/openai/00-check-key.ts`: print `describeMode()`; in azure mode print the
   endpoint, the normalised base URL, the deployment, masked tenant and client ids (first 4 and
   last 4 characters) and whether the secret is set, never its value. Exit if the deployment is
   missing. In service-principal mode first call `await azureTokenProvider()()` and print
   "entra token: OK" with the token length and elapsed ms (on failure: `explainError`, exit 1).
   Then one `generateText({ model: realModel(), prompt: "Reply with exactly the two letters: OK" })`
```

```
   and print reply, elapsed ms and `r.usage.inputTokens` / `outputTokens`.
   In openai mode keep the old behaviour. With `MOCK=1` print a note and exit 1.
4. Edit `labs/openai/03-server.ts` and `labs/openai/04-decision-provider.ts`:
   - delete the `createOpenAI` import; import `realModel` (and `describeMode` in 03) from
     `./_model` instead of `MODEL_ID`;
   - replace `createOpenAI({ apiKey: process.env.OPENAI_API_KEY ?? "" })(MODEL_ID)` with
     `realModel()`;
   - replace the `if (!MOCK && !process.env.OPENAI_API_KEY) {...exit...}` guard with
     `if (!MOCK) realModel(); // fail fast if the provider config is missing`;
   - in 03, the GET / health response uses `mode: describeMode()`.
5. Do not touch `01-file-agent.ts`, `02-deploy-agent.ts`, `03-client.ts`.
6. Update `labs/openai/README.md`: provider order (MOCK, then AZURE_OPENAI_ENDPOINT, then
   OpenAI) and the Azure variables: `AZURE_OPENAI_ENDPOINT`, `AZURE_OPENAI_DEPLOYMENT`,
   `AZURE_TENANT_ID`, `AZURE_CLIENT_ID`, `AZURE_CLIENT_SECRET`, optional
   `AZURE_OPENAI_API_KEY`, optional `AZURE_OPENAI_SCOPE`; note the required role.
## Constraints
- Keep every line under 100 characters. Keep `MOCK=1` behaviour identical.
- Never print, log or write a secret. No new files. No changes outside `labs/openai/`,
  `package.json` and `bun.lock`.
- Show me the full diff before applying it.
## Acceptance (run all of these; all must pass)
```bash
bunx tsc --noEmit --strict --skipLibCheck --target es2022 --module esnext \
  --moduleResolution bundler --types bun --allowImportingTsExtensions labs/openai/*.ts
MOCK=1 bun run labs/openai/01-file-agent.ts                # ends with "blocked calls: 2"
MOCK=1 AUTO_APPROVE=1 bun run labs/openai/02-deploy-agent.ts
MOCK=1 bun run labs/openai/04-decision-provider.ts         # ends with "status":"completed"
# wiring test with FAKE credentials: must fail with AADSTS90002 and the friendly explanation
AZURE_OPENAI_ENDPOINT=https://x.openai.azure.com AZURE_OPENAI_DEPLOYMENT=d \
AZURE_TENANT_ID=11111111-2222-3333-4444-555555555555 \
AZURE_CLIENT_ID=66666666-7777-8888-9999-000000000000 AZURE_CLIENT_SECRET=x \
bun run labs/openai/00-check-key.ts
```
EOF
```

Paste everything including the last EOF line.

End of guide.